Enhanced Phishing Website Detection Using Ensemble Machine Learning Techniques with Advanced Feature Extraction and Classification

A PROJECT

Submitted by

Meem Ibna Ashraf (2023809271)
Mohd Uzair (2021503834)

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Information Technology

Under the supervision of

Mr. Mohit Kumar

Assistant Professor

Department of Computer Science & Applications

Sharda School of Computing Science & Engineering (SSCSE)

SHARDA UNIVERSITY, GREATER NOIDA – 201310

May, 2026

TABLE OF CONTENTS

DECLARATION **1**
CERTIFICATE **2**
ACKNOWLEDGEMENT **3**
LIST OF TABLES **4**
LIST OF FIGURES **5**
SYMBOLS AND ABBREVIATIONS **6**
CHAPTER 1: INTRODUCTION **7**
1.1 INTRODUCTION AND OVERVIEW **8**
1.1.1 Motivation: **10**
1.1.2 Overview of Phishing Detection System **11**
1.1.3 Expected Outcome **12**
1.1.4 Gantt Chart **12**
1.1.5 Possible risk **14**
1.1.6 Software Requirement Specification (SRS): **15**
1.2 ORGANIZATION OF THE REPORT **16**
CHAPTER 2: RESEARCH METHODOLOGY **18**
2.1 Domain Identification(Cybersecurity & Phishing Detection) **19**
2.2 Problem Identification **21**
2.3 Technology Identification **23**
2.4 System view **25**
2.5 System Components & Functionalities **27**
2.6 Data & Relational views **28**
CHAPTER 3: PHISHING DETECTION SYSTEM: DESIGN CRITERIA **30**
3.1 Introduction **31**
3.2 Background **33**
3.3 System Design **35**
3.4 Design Diagrams(DFD, Use Case, Sequence) **36**

3.5 System Architecture **41**
3.6 Information & Communication Design **42**
CHAPTER 4: DEVELOPMENT & IMPLEMENTATION OF THE SYSTEM **43**
4.1 Introduction **44**
4.2 Related work(Literature Review) **47**
4.3 Developmental feasibility **48**
4.4 Implementation Specifications **51**
4.5 Training and Evaluation **53**
4.6 Critical modules of System **55**
CHAPTER 5:MACHINE LEARNING & ENSEMBLE-BASED PHISING CLASSIFICATION **57**
5.1 Introduction **58**
5.2 Background **59**
5.3 Coding and Implementation **61**
5.4 Efficiency and Output **64**
5.5 Conclusion **65**
CHAPTER 6: RESULTS & TESTING **67**
6.1 Result **68**
6.1.1 Success cases **71**
6.1.2 Failure cases **72**
6.2 Testing **72**
6.2.1 Type of testing adapted **73**
6.2.2 Test results of various stages **74**
6.2.3 Conclusion of Testing **77**
6.3 Success of System **78**
CHAPTER 7: CONCLUSION & FUTURE DIRECTIONS **81**
7.1 Performance Estimation **82**
7.2 Usability of System **83**
7.3 Limitations **83**
7.4 Scope of Improvement **84**
LIST OF PUBLICATIONS **86**

CERTIFICATE……...88
REFERENCES **90**
GitHub repository link……...92

APPENDIX

A. Code Snippets……..93
B. Additional Data……100

DECLARATION

I hereby declare that the project work entitled "Enhanced Phishing Website Detection Using Ensemble Machine Learning Techniques with Advanced Feature Extraction and Classification" submitted to Sharda University, Greater Noida is a record of an original work done by us under the guidance of Mr. Mohit Kumar and this research is being submitted to fulfil the requirements for the award of degree of Bachelor of Technology in Computer Science and Applications.

The results embodied in this research work have not been submitted to any other university or institution for the award of any degree or diploma.

Place: Greater Noida Signature of the Student-1

Date: Signature of the Student-2

CERTIFICATE

This is to certify that the report entitled **"Enhanced Phishing Website Detection Using Ensemble Machine Learning Techniques with Advanced Feature Extraction and Classification"** submitted by Mr. Meem Ibna Ashraf(2023809271) and Mohd Uzair(2023809271) to Sharda University, towards the fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Information Technology**, is a record of bonafide final year project work carried out by them in the Department of Computer Science and Applications, School of Engineering and Technology, Sharda University.

The results/findings contained in this project have not been submitted in part or full to any other University/Institute for the award of any other Degree/Diploma.

Mr. Mohit Kumar

Assistant Professor

Dr. Rajneesh Kumar Singh

Head & Associate Professor

Place: Sharda University

Date:

Signature of External Examiner

Date:

ACKNOWLEDGEMENT

A major project is a golden opportunity for learning and self-development. We consider our self very lucky and honored to have so many wonderful people lead us through in completion of this project.

First and foremost, we would like to thank **Dr. Rajneesh Kumar Singh, HOD, CSA** who gave us an opportunity to undertake this project.

Our grateful thanks to **Mr. Mohit Kumar** for his guidance in my project work. Mr. Mohit Kumar, who despite being extraordinarily busy with academics, took time out to hear, guide and keep us on the correct path.

CSA department monitored our progress and arranged all facilities to make life easier. We choose this moment to acknowledge their contribution gratefully.

Signature of Students

Meem Ibna Ashraf (2023809271)
Mohd Uzair (2021503834)

LIST OF TABLES

Table No.	Title	Page No.
Table 2.1	Comparison of Phishing Detection Techniques	24
Table 3.1	Functional Requirements of the System	39
Table 3.2	Non-Functional Requirements of the System	40
Table 5.1	Technology Stack Used in the Project	63
Table 6.1	Performance Metrics of the Model	70
Table 6.2	Test Cases and Results	74
Table 6.3	Testing Summary of the Phishing Detection System	78

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 1.1	Figure showing the visits made, representing physical Gantt Chart	13
Figure 2.1	Methodology Workflow of Phishing Detection System	27
Figure 3.1	Data Flow Diagram (DFD Level 0)	36
Figure 3.2	Data Flow Diagram (DFD Level 1)	37
Figure 3.3	Use Case Diagram	38
Figure 3.4	Sequence Diagram	39
Figure 3.5	System Architecture Diagram	41
Figure 4.1	User Interface – Login Page	46
Figure 4.2	User Interface – Dashboard	53
Figure 4.3	URL Detection Result Page	55
Figure 4.4	Chat-bot Interface	56
Figure 5.1	Methodology Workflow of the Proposed Phishing Detection System	61
Figure 6.1	Accuracy Comparison Graph	70
Figure 6.2	Confusion Matrix of the Proposed Model	73
Figure 6.3	Precision vs Recall Graph	76

SYMBOLS AND ABBREVIATIONS

ML	Machine Learning
DL	Deep Learning
URL	Uniform Resource Locator
API	Application Programming Interface
RF	Random Forest
SVM	Support Vector Machine
NB	Naive Bayes
KNN	K-Nearest Neighbors
CNN	Convolutional Neural Network
RNN	Recurrent Neural Network
SSL	Secure Sockets Layer
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
WHOIS	Domain Registration Lookup Service
FPR	False Positive Rate
TPR	True Positive Rate
AI	Artificial Intelligence

CHAPTER 1:

INTRODUCTION

CHAPTER 1:
INTRODUCTION

1.1 INTRODUCTION AND OVERVIEW

The Internet technology has experienced an explosive growth over the last decade transforming the way we live, work, and operate as citizens. Digital technology has revolutionized the way we interact, process, and manage our daily lives. Services like online banking, cashless transactions, e-commerce, cloud storage and computing, telecommuting, online learning, and many others have become an essential part of our daily lives. As we reap the benefits of digital technology, we are also exposed to new security threats.

The Internet has become an invaluable medium of communication as Internet users have grown in numbers. However, this shift has brought along a corresponding surge in the transfer of more sensitive information over the Internet including personal details, financial information, login credentials, and other proprietary information that organizations have shared with their clients. Valuable as this information is to cybercriminals, the number of threats have been growing at an exponential rate. Of all the threats facing Internet users and organizations, phishing is the most prevalent, most critical, and most damaging. Phishing is a method used by cyber attackers to extract or illegally access sensitive and personal information of users by pretending to be a trusted entity such as bank, e-commerce company, social media or government organizations etc. Majority of phishing attacks are delivered via e-mail to a pool of email addresses and occasionally via SMS, social media and websites that appear to be legitimate but are fake. Phishing is different from other types of cyber attacks that rely on vulnerabilities of systems and applications etc. Phishing attacks exploit human psychology to entice users to take a specific action at a time of uncertainty or trust. They create a perception that users are going to loose out on a great offer if they do not act quickly. Users are lured into clicking on malicious links, downloading malicious files or divulging sensitive information including usernames and passwords. This high quality phishing emails are so effective is primarily due to their high quality imitation of online brand web properties, including accurately replicating branding such as logos, templates, font and colour schemes used on the website. The hackers tend to amend the domain name in the URL bar to include additional details, yet generally remain very similar to the genuine site. Due to this, it can be tricky for online users to distinguish between the two, particularly to those unfamiliar with the tactics and under pressure. Phishing, once a pretty basic form of online hacking, has evolved to include far more sophisticated scams. Gone are the days of easy-to-spot emails riddled with grammatical errors, incorrect URLs and poorly designed landing pages. The most recent phishing trends show that hackers are at it again, and this round, they're using tricks that are far sneakier. Here are some of the most recent phishing email scams and how to protect yourself from them:

•**URIDeception:** This type of threat is also referred to as domain spoofing and works by a web application being tricked into following a URL to an entirely different site by using a false domain.

•**Plaintext dump:** a specific scenario where unencrypted information is inadvertently shown when security benchmark output is crafted to resemble a plaintext dump.

•**URL Obfuscation** using URL shortening services such as TinyUrl to conceal malicious code.

•**HTTPS Misuse:** Using secure certificates to gain user trust.

•**IP Spoofing:** SNMP messages are relayed from a spoofed IP address allowing the packets' origin to be masked.

These techniques make phishing detection extremely challenging, even for experienced users.
Even after a phishing campaign has been detected, a number of other challenges remain. Most phishing websites are published within a few minutes and live only for a few days. Within this short time span, the attacker tries to capture as many victims as possible. Static-based detection approaches are insufficient for such fast-changing phishing websites. Traditional phishing detection systems are based on blacklist-based systems to a significant degree, which, practically, place identified phishing URLs into a database and block them when they are accessed by users. This is actually an effective way in fighting the threats which are already identified however, it is not as effective as the revelation of the newly set up phishing websites also known as the zero-day attacks. The blacklist-based detection is no longer sufficient due to the high rates of the creation of new domains and the change of methods used by phishing attackers. The detection abilities have been greatly improved using machine learning techniques. Machine learning can be applied on vast datasets and determine patterns that can differentiate between a phishing and legitimate website. The duration of the URL, age of the domain, utilization of the special characters and URL structure is commonly used to make appropriate predictions in these models. Machine learning has a number of benefits over traditional methods. It enables automatic detection and adaptation to evolving threats and learning on historical data. Nonetheless, one machine learning model might not be the most effective. It does not necessarily perform well and performance and classification errors such as false positives and false negatives may be encountered. Ensemble learning techniques are used to overcome these limitations by integrating them in phishing detection systems. To enhance the accuracy and reliability of prediction, ensemble methods use several machine learning models. The strengths of the different classifiers will enable the system to minimize errors and generate stronger results. This is particularly useful in phishing detection whereby various models might be effective under varying data characteristics. The other important aspect of the system is a rule-based analysis. It applies pre-defined parameters to detect suspicious patterns of URLs, such as the application of misleading keywords, the use of an unusual number of special characters or abnormal URLs. The rule-based analysis and the machine learning contribute to the increased accuracy and interpretability of the system. In addition, real-time integrated threat intelligence is important in boosting the detection of phishing. The system can connect both to external APIs and to verify URLs on the fly as well as have access to continuously updated databases of known phishing sites.

This enables the detection system to detect emerging threats much better and be responsive in a dynamic threat environment. The suggested system is based on a hybrid approach that will be a combination of machine learning, ensemble learning, rule-based analysis, and real-time API integration. It is carried out in the form of a web-based application, which enables the user to provide the URLs and be offered instant feedback on whether the site is secure or can be a malicious one. The overall system design comprises modules that receive the input, extract features, classify with machine learning, decision making with the help of ensembles, rule-based validation and API integration. The modules work together to have efficient, reliable and accurate phishing detection. The system also emphasizes the usability since it has a clean and user-friendly user interface. Users have the option of adding URLs easily and are provided with clear results on detecting results. Besides, the system will be scaled and performance driven so that it can be able to face multiple requests at a time without affecting the response speed. These attributes make the solution suitable to be used in a real-life situation.

1.1.1 Motivation

The idea to develop this project was motivated by the fact that phishing attacks become rapidly developed and have severe outcomes on individuals and companies. Nowadays, it is commonplace that Internet users are carrying out different operations using online services. Individuals bank, shop, communicate and store their data in the cloud. This development has exposed users to cyberattack, particularly phishing attacks. The initial reason why I am inclined to take part in research work is that Internet users are unaware of phishing tricks. Many people are not able to differentiate between fake webpages and websites provided by legitimate companies. Phishing sites normally look exactly like the authentic ones and thus, individuals willingly provide personal information such as login and other important fragments of data. Secondly, the lack of efficiency of existing phishing detection systems motivates it. As an example, most of the tools that are applied to defend against cyber threats utilize a blacklist where known malicious links are held. Although it is a good strategy to counter the known threats, this strategy fails to safeguard users against newly-generated phishing sites. There are some difficulties associated with rule-based systems too. The thing is that some of the conditions are pre-determined and are not able to embrace all the possible variants of actions of the attackers. In addition, advanced strategies will result in false positives and false negatives. But nowadays, with the development of technologies, it is possible to use machine learning algorithms in cybersecurity. Such methods can be used to derive different data and trends hence forecasting future behavior of websites. Nevertheless, different machine learning models might provide different results, depending on the dataset and the set of features. This issue can be alleviated with the help of ensemble learning procedures that can train multiple models on various data subsets. Thus, it is possible to ensure high prediction rates by combining the benefits of each particular machine learning model. Besides, phishing websites are to be identified instantly as one cannot wait until the page is loaded.

Moreover, people cannot do without this process as they can obtain an immediate reaction following typing in the URL address. It is also necessary to develop user-friendly software as there are numerous existing solutions that are rather complicated to use. Thus, the most important incentive of my research activity is to create the effective and high-tech phishing recognition system.

1.1.2 Overview of Phishing Detection System

The proposed phishing detection system is to be an intelligent, multi-layered and all-purpose security system and will involve the use of a mixture of sophisticated methods to detect dangerous websites successfully. The contemporary dynamic digital world is becoming more sophisticated and intricate when it comes to phishing attacks, and that is why these attacks cannot be identified in the same manner. To solve this, the system uses a hybrid system, which combines machine learning, ensemble techniques, rule-based analysis, and real-time threat intelligence to improve on reliable and accurate detection. The very mechanism of the system works on the principle of analyzing the URLs typed by users, and determining a set of features to recognize a legitimate and potentially malicious web site. These features are selected based on the studies and real-life observations of phishing. The system is a modular system where each component can undertake a specific task and therefore it is flexible, scaled and easier to maintain. It begins with the Input Module which is the entry point. To check a URL, the user enters a URL he wants to check and the system checks the URL by first determining that it is well formed and contains no invalid characters. This will avoid mistakes and the system will maintain a smooth operation. After the URL has been validated, it is passed to the Feature Extraction Module that can be regarded as one of the most important components of the system. In this case, various forms of features are mined, such as lexical, structural, and domain-based features. Lexical features involve the length of URL, use of special characters, suspicious keywords and use of numbers. The structural features are related to the structure of the URL, the subdomains and the components of the path. Domain based features allow such information as domain age, registration and WHOIS information. All these features come up to create an in-depth understanding of the site. These features, which are extracted are then fed into the Machine Learning Module to be classified. This module uses different algorithms and they are, the Random Forest, Support Vector Machine (SVM), Naive Bayes, and K-Nearest Neighbors (KNN). The two models are trained using data sets of both legitimate and phishing URLs, allowing the models to predict and detect trends. The system also includes Ensemble Module which combines the predictions of other models to make them more accurate. It does not depend on one model alone but instead employs the techniques such as voting or weighted averaging in order to make a final decision. This method minimizes mistakes and enhances reliability. The system also has a Rule-Based Module to detect the known suspicious patterns besides machine learning. These guidelines are outlined using typical phishing clues, including unfamiliar domain constructions, excessive special characters, or unsettling keywords. This provides an additional level of detection as well as simplifies the results interpretation. The system also has an API Module of real time threat intelligence. It is able to match URLs with continually updated lists of malicious web sites by making contact with external services like the Google Safe Browsing API. This is to make sure that even phishing sites that are created recently can be detected. Lastly, the Results are displayed in an easy-to-understand manner by the Output Module. It informs the user on the safety or even the possible risk of the URL and can also give some additional information to support the decision to allow

users to make an informed decision choice. On the whole, this system is based on a layered and hybrid method to detect known and emerging phishing threats. Its modular design also renders it scalable, adaptable as well as real world applicable.

1.1.3 Expected Outcome

The primary objective of this project is to come up with a system of detecting phishing sites that is not only effective but also precise and reliable in detecting malicious sites. In this way, it will safeguard users against all sorts of cyber threats and address the inefficiency of the current detection solutions by having a more integrated and holistic approach. One of the key expected outcomes is achieving a high level of accuracy in classification. The system will be proposed to be able to differentiate between legitimate and phishing websites in a proper manner and minimize misidentification, such as false positives and false negatives. The false positives (safe sites are labeled as malicious) and false negatives (harmful sites are not detected) both potentially impact user trust and thus reducing these is critical. The other significant consequence is the real-time threat detection capability. The system is designed to give immediate responses to users as they browse as the users need to know the safety of a site within a second. This is backed by the incorporation of external APIs that provide up to date information about known threats to keep the system up to date. The project is also aimed at raising the awareness of users. The system provides clear and informative results to users, giving them the reason why a site is risky. Not only does this build trust but users are also educated on how to be more aware of phishing in future. Another priority is to improve overall performance. Several requests are efficiently managed by the system, which is fast and easy to use. It is also scalable, i.e., it can be deployed within a large user environment without any performance problems. Adding ensemble learning should also contribute to the strength of the system. The system can also be more versatile in identifying various phishing strategies as it can consist of various machine learning models, resulting in a higher accuracy and consistency than when individual models are used. Besides, the system will be user-friendly and approachable. Having a simple and intuitive interface, anyone can easily interact with it without the need to have technical expertise which promotes broader usage. In sum, the project will provide a workable, scalable, and effective phishing detection system that can be implemented to the real world to enhance cybersecurity and safeguard users.

1.1.4 Gantt Chart

The creation of the phishing detection system is a well-planned and structured process, which ensures that every phase of the project is executed effectively and punctually. Planning contributes significantly to the success of the project, and the tools such as the Gantt chart can be used to visualize the plan and manage it effectively. The project is broken down into a number of phases each of which concentrates on a particular aspect of the process of development. It starts with identification of the problem and research where the scope of the projects is outlined and the literature is examined. The step assists in knowing the existing solutions and what should be improved. The second step is directed at the data collection and preprocessing. During this phase, phishing and legitimate URLs datasets are gathered using trustworthy resources. The data

is cleaned and made ready so that it is consistent and can be analyzed. The main section of the project is the extraction of features and the development of the model. The data is extracted to obtain relevant features and various machine learning models are trained and tested. Different algorithms are tested to assess the best algorithms in detecting phishing. This is followed by the system design phase during which the general architecture of the system is designed. Data flow diagrams (DFD), use case diagrams and sequence diagrams are some of the diagrams that are drawn to have a clear picture of how the system will operate. The implementation stage comes next, in which the real development occurs. These involve system coding, assembling various modules, developing the backend, creating the frontend, and linking APIs. After constructing the system, a testing and assessment is done to confirm that all is well. The functionality, accuracy and performance of the system are checked using different test cases. Lastly, documentation and report writing are done to conclude the whole project and its results. On the whole, the Gantt chart gives a clear picture of all these stages, which assist in tracking progress, to manage time and to ensure the project remains on schedule.

📌 ☞ *(Add Figure: Gantt Chart here)*

1.1.5 Possible Risks

Every software development process entails some risks or uncertainties. It becomes necessary to identify the potential risks at the beginning of the process for smooth implementation. In a phishing detection system development, the risks can be technical, operational, and security related as well. Being associated with cybersecurity, any small problem could impact the performance of the software. The issue of poor data quality and unbalanced datasets probably constitutes the greatest risk of this project. The data in the dataset will be relied on to ensure that the machine learning algorithm operates properly. In case of the existence of any wrong data in the data set, then the algorithm will fail. While detecting phishing sites, it is necessary to have a balanced set of both phishing sites and legitimate ones. An unbalanced set will, however, give the algorithm inclination towards a side at the expense of the other, which may cause phishing sites to be misidentified. The second risk factor relates to the model's overfitting and underfitting. Model overfitting happens when the performance is exceptionally high on the training data set but extremely low on unseen data. This takes place when the machine learning algorithm learns the data, instead of understanding its patterns. Conversely, underfitting is when a machine learning model does not have sufficient complexity to understand patterns in the data. In this case, it becomes difficult to implement a system in real life. This means that adequate measures should be taken to mitigate this issue. In addition, the third risk is related to the system's dependency on external APIs for collecting information about potential threats. This brings certain dependency risks. These risks are associated with issues such as API unavailability, malfunctioning, wrong data provision, network problems, or API rate limits. This is especially important for the system since the real-time detection of possible threats is its critical feature. Yet another crucial risk concerns the performance and scalability of the system. As more users become active on the platform, it needs to process numerous requests concurrently. Otherwise, it can suffer from sluggishness, lagging, and even complete failure under heavy load. In order to avoid such situations, it is necessary to optimize the system and manage its resources efficiently. Moreover, security risks are among the most significant threats to the system. Indeed, since it processes user input by receiving URLs from them, it becomes vulnerable to attacks aimed at the insertion of malicious code or corrupted data. Also, any improper processing of user data might lead to the violation of their privacy rights. The second threat is associated with the fact that methods employed by attackers to avoid detection tools are constantly evolving. This will render the upkeep of accuracy hard in the long term. An effective model today may fail in the future as its methods will become obsolete with the appearance of other kinds of threats. Thirdly, there are possible risks associated with false positives and false negatives. The former would result in blocking of legitimate sites, which would be inconveniencing and the latter will result to phishing sites not being detected by the system, hence some security problems. It is significant to balance these two issues. The mitigation strategy in responding to the risks entails data preprocessing techniques like data cleaning, data normalization and balancing. The cross-validation techniques are used to validate to ensure that the model can be generalized. The model is constantly being updated and revised to meet changes in the environment. Fallback methods are used in case of an API failure. Input validation and error handling is used to maintain security.

1.1.6 Software Requirement Specification (SRS)

The software requirement specification (SRS) is a mandatory document that provides details on the requirement and expectations of the anti-phishing system. It is a plan of the development process and provides the insight on what the system should be able to do. The software requirement specification document makes sure that all the actors in the processes of evaluation and development know about the system capabilities and functions. There are two types of requirements outlined in SRS, namely functional requirements and non- functional requirements.

- **Functional Requirements**

Functional requirements encompass the basic functions that the system has to fulfill. They explain how the system interacts with the users, and data processing. The system has to enable the user to enter the URL. The most important functionality is this one since the entire system is constructed on the basis of phishing URL detection. The input module will have to make sure that the data that will be entered is formatted correctly. After the URL has been received, it is time to extract the appropriate features. These features are the lexical features of the URL such as the length of the URL, the presence of special characters, the number of subdomains, domain features, such as the age and registration information of the domain. The extraction of features is crucial in giving the machine learning models the capacity to identify patterns. The system has to utilize machine learning models to categorize the URL as phishing or legitimate. This classification should be accurate and on the trained models. Ensemble techniques may also be included in the system to enhance accuracy of predictions. The results should be displayed to the user in the proper way. Output has to inform the user if the website is safe or malicious. Additional details could be added as well. Lastly, the other vital functional requirement is in regards to the integration of real-time APIs. The system should be able to verify the entered URLs with different threat intelligence databases. This will enable the detection of new phishing websites.

- **Non-Functional Requirements**

The quality standards that the system has to satisfy are known as non-functional requirements. Their concerns are the system performance, ease of use, reliability, and safety. Two of the most crucial things to do during system implementation are trustworthiness and consistency. The system should work with few problems, which makes sure that the results are accurate and reliable. Performance is another imperative. The system is expected to be fast in responding thus people can receive results immediately. Otherwise, there will be a poor user experience. Scalability is also another consideration that should be made. The system should be in a position to support additional users without making the system slow, and this is critical when it is going to be used in the real world.

Efficacy of the system is largely dependent on the usability. It must have easy to use interface,

which enables a user to interact with it without technical knowledge. Security the system is highly sensitive in terms of security. It must be capable of ensuring safety in how user data is handled as well as the lack of unauthorized access. To minimize the possible vulnerabilities, measures like input validation and adequate error handling should be adopted. Availability and reliability are also imperative. The system should be reliable and not in frequent breakdowns and should be available when the users need it. The SRS document consolidates all these requirements, making them well defined, and well responded to during the development process. It can be used as a road map to design, construct and test the system. Overall, Software Requirement Specification is necessary to ensure that the phishing detection system will meet the expectations of users and will be efficient in real-life scenarios.

1.2 ORGANIZATION OF THE REPORT

The report of the Phishing Detection System project is logically divided into seven chapters, which deal with a particular part of the work. Such an organised methodology ensures smooth and rational flow of information, leading the reader through the introduction of the problem at the beginning, and its possible future developments and suggestions. This type of structure simplifies the report.

Chapter 1: gives a summary of the whole project, introducing the basic ideas revolving around phishing attacks and cybersecurity. It provides the context to the issue, including the increasing danger of phishing in the modern online world. The other aspects discussed in the chapter are the impetus behind the project, the necessity of more sophisticated detection techniques and the constraints of the older ones. Also, it describes the anticipated results, possible risks, and the Software Requirement Specification (SRS) that provide an in-depth understanding of the system and its purposes.

Chapter 2: Research Methodology is dedicated to the methods and approaches followed in the project. It starts with the domain identification, situating the work in the area of cybersecurity and phishing detection. The chapter then describes how the identification of the problem occurs, which has complications in identifying phishing websites. It also entails technology identification whereby, the tools, frameworks and programming languages are addressed. In addition, it provides the system architecture, components and functionalities, data and relational views giving a clear picture of how the system processes and manages information.

Chapter 3: Phishing Detection System Design Criteria is concerned with the design of the system. It includes the background and important considerations that should be taken into account when formulating an efficient phishing detection model. This chapter describes the system architecture and gives several design diagrams such as Data Flow Diagrams (DFD), Use Case Diagrams and Sequence Diagrams to depict how the system works and how it interacts with other systems. It also touches on information and communication design to make sure the system structure is well defined and that it works effectively.

Chapter 4: Development and Implementation of the System is concerned with the practical implementation of the proposed system. It starts with the description of the development process, as well as the review of the literature to explore available solutions and their shortcomings. Another concept in the chapter is developmental feasibility, which is to make sure that the system is technically and economically feasible. Moreover, it gives specifications of how the implementation should be carried out in detail, such as which tools, technologies, and structures will be employed. Moreover, it discusses how machine learning models are trained and assessed and outlines the major modules that constitute the system.

Chapter 5: Machine Learning and Ensemble-Based Phishing Classification describes the principles of work of the system. It dwells upon the use of machine learning algorithms and ensemble methods on the competent detection of phishing. The chapter explains different models, such as Random Forest, Support Vector machine and Naive Bayes and their application in classification. It also sheds light on the use of ensemble learning in integrating various models to enhance accuracy and minimize errors. Besides, details of coding and implementation are also discussed, as well as the efficiency of the system and analysis of the generated outputs.

Chapter 6: Results and Testing, reports the evaluation of the system by testing it thoroughly. It incorporates the findings of the implementation of the system to various datasets and situations. The accuracy, precision, recall, and F1-score are examined as key performance metrics. Both successful and unsuccessful cases are also discussed in the chapter and the strengths and limitations of the system have been identified. Different testing procedures are described, and specific results in each stage are given in detail to give a comprehensive evaluation of system performance and reliability.

Chapter 7: Conclusion and Future Directions represents a summary of the whole work and points the key findings of the project. It will assess the efficiency of the system and how applicable it is in real life. Current limitations are also outlined in the chapter and improvements that could be made are proposed. The subsequent improvements can be anticipated: more sophisticated machine learning models, better feature extraction methods and other security measures can be used.

In general, the report is well structured and logical and therefore clear and coherent. The chapters are structured sequentially and provide a thorough insight into the phishing detection system, starting with the idea, its implementation, and assessment.

CHAPTER 2:
RESEARCH METHODOLOGY

CHAPTER 2:
RESEARCH METHODOLOGY

2.1 Domain Identification (Cybersecurity & Phishing Detection)

To begin with, the area of this project lies under the field of Cybersecurity and specifically phishing websites detection, which is one of the most prominent spheres of digital security nowadays. Due to the recent high growth in technology, internet-based systems have been in use at a very high rate. In the modern day, individuals and organizations rely on the internet to conduct their activities including banking, e-commerce, communication, storage of data and cloud services among other activities. Although this move has come with numerous benefits, it has also increased vulnerability to cyber threats. Cybersecurity is the security of systems, networks, and data against unauthorized access, attacks, and possible damage. With the constantly changing nature of cyber threats, the significance of proper cybersecurity measures has increased tremendously. Phishing is now one of the most common and dangerous threats out of the many types of cyberattacks. It poses a special threat as it is more focused on human behavior rather than technical vulnerabilities and is more difficult to track and eliminate. Phishing is a type of social engineering attack where an attacker pretends to be a legitimate party and tricks the user into providing sensitive information. Such attacks are usually carried out via fake websites, emails, or messages which look authentic. As an example, a hacker can create a rogue banking site and make it look like the original one and deceit the users to provide their logins. Once received such information may be abused in activities like financial fraud, identity theft and other evil intentions. The possibility to take advantage of human psychology is one of the main reasons why phishing attacks are so effective. Attackers usually instill a sense of urgency or fear and may prompt users to take immediate action without first checking the source authenticity. As an illustration, one may get a notification that their bank account will be blocked unless they update their information immediately. These tactics greatly contribute to the likelihood of users becoming victims of phishing attacks. Phishing detection - methods for detecting and preventing such malicious practices. Phishing detection techniques. Phishing detection has traditionally focused on implementing blacklist-based methods. These methods rely on known phishing URLs, that have been identified and detected previously, and store them in a database. The system then proceeds to block these identified threats. However, these approaches are easy to implement and are very efficient for known threats; however, newly created phishing websites cannot be detected because blacklists will be obsolete after a while. After a while attacker will create enough domains and new to be detected phishing websites.

In addition to the methods mentioned above, patterns can also be checked against a set of predefined rules. This method is commonly used to detect known types of spam, and works by defining rules based on features such as URL patterns with special characters, domain names that appear to be spoofed, or keywords that seem suspicious. While simple to implement, this method has its own set of limitations. Rigid rules are not designed to handle unusual attacks, and are easily defeated by new methods of spamming. Recent studies have looked to apply machine learning to tackle the problem of Phishing Web Page detection. By utilising machine learning algorithms, vast amounts of data can be processed to identify features which can distinguish between a phishing website and a safe one. These models are trained using features extracted from URLs and generate predictions based on the patterns learnt from said features. Machine learning techniques therefore provide a more dynamic and robust solution to the problem that has previously been addressed by static methods. This project aims at identifying phishing web pages using a combination of machine learning techniques. In this project, I have tried a combination of approaches and used multiple classifications algorithms : (Random Forest, Support Vector Machine, Naive Bayes, K-Nearest Neighbors to train a model using different data sets.) To boost the performance of the project, I have been experimenting with ensemble learning techniques that aggregate the predictions of multiple models together to obtain better results than individual models. Ensemble methods aim to improve accuracy by making the best use of the strengths of individual models while reducing their respective errors. This is particularly useful in email phishing detection, where various approaches may achieve greater accuracy on different types of datasets. The product incorporates Machine Learning Technology into its phishing detection, and in addition includes rule-based phishing detection as well as real-time API URL verification against the latest malicious site databases. Another bite taken out of the apple of phishing detection. But there are a lot of other bites that have already been taken from this apple, and this one really doesn't amount to much

• URL Analysis – we look at the organization of the URL as well as the content, checking for any suspicious activity. We look at the URL length, presence of special characters and number of subdomains. We also check if the URL contains https or not.

• Our software extracts features from the URL and puts them in a format that can be used by a machine learning model to predict whether a URL is malicious or not. Feature extraction is an important component to achieving good accuracy with this type of work.

• Classification Algorithms: These are special machine learning algorithms, used to classify URLs (such as www.google.com) into two categories (Phishing / Legitimate), to determine whether a given URL is phishing or not. These algorithms are trained on existing data, which has been manually classified by the administrator. There are several classification algorithms; they all function differently, and their results may vary accordingly.

• Threat Intelligence Systems - These systems provide real-time data about known phishing sites, and other current threats, which can greatly improve detection if the system has been updated with threat intelligence data.

Phishing is not a static crime, phishers evolve their tactics on a daily basis to circumvent detection by current anti-phishing approaches. In this paper, we demonstrate that techniques developed in 2006 are no longer sufficient for dealing with the phishing scenarios in 2008. Phishers are exploiting URL shortening services, replicating URL manipulation in multiple web pages, and abusing HTTPS to trick users into revealing sensitive information such as passwords and personal identifiable information. In this paper, we present a set of techniques and a system to automatically detect phishing Web pages.

The ability to detect web site attacks in real time is discussed in this research project. Real-time processing of data is becoming increasingly important, as users of websites can expect immediate results. In some cases, even a slight delay in processing can have severe consequences to the user, such as losing money or having their data stolen. Results are obtained quickly and accurately. The domain also focuses on making sure users are aware of what's going on and that the system is easy to use. If people can't understand how to use a detection system, it's not going to be very helpful. So, the system gives clear signals about whether a website is safe or not, which helps users make good choices. This way, users can stay safe online by knowing which sites to avoid. This project is a big deal for cybersecurity. It brings together a bunch of different techniques to make a strong system for catching phishing scams. The old ways of doing things had some flaws, but this new system is more reliable. By using machine learning, ensemble techniques, rule-based analysis, and real-time APIs all together, it's both good at its job and easy to use in real life. This is important because phishing is a big problem that can cause a lot of harm. The project's approach is new and exciting, and it could make a real difference in the fight against cyber threats. In conclusion, the domain of cybersecurity, particularly phishing detection, plays a vital role in ensuring the safety of online users. With the increasing number of phishing attacks, there is a growing need for intelligent systems that can detect and prevent such threats. This project aims to contribute to this domain by developing a comprehensive phishing detection system that enhances security, improves accuracy, and provides real-time protection.

2.2 Problem Identification

The rapid increase in phishing attacks has become a major concern in the field of cybersecurity. With the growing dependence on internet-based services such as online banking, e-commerce, and digital communication, users are constantly exposed to cyber threats. Among these threats, phishing attacks are particularly dangerous because they exploit human behavior rather than technical vulnerabilities. As a result, even well-secured systems can be compromised if users are deceived. One of the primary problems in phishing detection is the dynamic and evolving nature

of phishing websites. Attackers continuously develop new techniques to bypass existing security mechanisms. Modern phishing attacks are no longer simple or easily identifiable. Instead, they are designed to closely mimic legitimate websites using advanced techniques such as domain spoofing, URL obfuscation, and HTTPS misuse. Domain spoofing involves creating domain names that are visually similar to legitimate ones, making it difficult for users to distinguish between real and fake websites. URL obfuscation hides the actual destination of a link, often using shortened URLs or encoded characters. Phishing emails are increasingly being laced with URLs that point to an https-enabled webpage set up by the attackers. This means the webpage is presented to the victim over HTTPS, making it appear to be more credible and thus more likely to be trusted. Besides phishing e-mails and pagekits, also content is hosted on phishing websites. This content is used for luring victims to login pages or to get banking credentials and other sensitive information. New phishing websites get published quickly and have a very short lifespan. As a consequence, detection systems are often overwhelmed by a short wave of samples and the used content is only active for a very short period of time. Usually, detection systems can block the content within a few minutes, after which the phishing website is taken offline. But the threat is growing and newly assembled phishing websites appear quickly. Current phishing detection solutions on the market, such as anti-phishing software, focus on the identification of phishing activity using a number of different methods. Many of these solutions employ a blacklist approach where the software checks whether the URL that the user is attempting to access has been previously identified as a phishing site and placed in a database. The software then blocks the user from accessing the site. This approach can be good for stopping known phishing threats, but it is not effective against zero-day attacks where the phishing site has been created and is "attacking" within a matter of hours. This means that the site has been registered as a new domain and is being used within a very short timeframe for phishing, often within hours. As a result, the approach is often outdated before it can be put into action. Similar limitations exist for rule-based systems that identify phishing emails based on predefined patterns or rules. In rule-based systems, emails are classified as phishing based on characteristics such as the presence of suspicious keywords, unusual URLs, or excessive use of special characters. Rule-based approaches are easy to deploy and implement, but they have a main drawback they are not designed to handle shifting patterns of phishing e-mails over time. In order to tackle the phishing problem by filtering out phishing web pages using machine learning methods, it is an important problem to achieve a proper error rate. Namely, while one can encounter a false positive, i.e., a legitimate web page is classified as phishing, or a false negative, i.e., a phishing web page is classified as legitimate. While both errors seem trivial at a glance, the false positive causes users discontent and lowers the system's usability and also affects credibility of web pages recommended by the system. On the other hand, a false negative could lead to critical security threats to users. The problem of real-time detection is also significant. In practical scenarios, users expect immediate feedback when interacting with websites. Delays in detection can result in users accessing malicious websites before they are identified as phishing. Therefore, a system must be capable of providing fast and accurate results in real time. Another challenge is the complexity of feature extraction. Identifying relevant features that can accurately distinguish phishing websites from legitimate ones is not straightforward. Features such as URL length, domain age, number of subdomains, and presence of special characters must be carefully selected and processed. Incorrect feature selection can reduce model performance.

In addition to the prior requirements, the system should scale well. A system that does well on a small scale doesn't necessarily translate to real-world usage as the number of concurrent users increases. Phishing detection systems, especially those based on machine learning approach, also face challenges to obtain high-quality datasets. Most of existing researches rely on limited number of datasets which are mostly created several years ago and suffer from low quality. Many of them are outdated due to the rapid evolution of phishing campaigns and therefore, are not effective in real- world scenarios. Using these datasets for training and evaluating models may result in biased models which generalize poorly to new, unseen phishing examples. Speak clearly and have people recognise my name within a few seconds at the start of each message and choose the right speaker even when I mumble or talk over others.

• Detect both known and unknown phishing websites

• Provide real-time results

• Minimize false positives and false negatives

• Adapt to evolving phishing techniques

• Handle large-scale data efficiently

We employ a combination of machine learning, ensemble methods, rule-based approaches and API exploitation to tackle these problems. Our spam filtering software uses Machine learning models that learn from the data and gets better over time by combining multiple models together to better predict spam. We can also enable obvious phishing rule based detection as well as integration with multiple threat intelligence API to receive a real time threat feed. The system incorporates several features and techniques to address the challenge of phishing web page detection and improve on current approaches.

2.3 Technology Identification

In this post we will be covering the stack of technologies, libraries and services that we used while building a system for phishing detection. The technology stack was chosen based on efficiency, scalability, ease of use and applicability to the problem at hand. I employed the primary programming language of Python for this project. Python is one of the main programming languages in the domain of data science and machine learning. Python is an easy language to read and it is extremely efficient at building complex systems. There is a huge community and a multitude of libraries and toolboxes that exist to provide pre-built functionalities to assist in tasks such as data analysis and building a machine learning model. The project uses Scikit-learn, a common Python library for machine learning, for training the machine learning models. This library offers a lot of algorithms like Random Forest, Support Vector Machine (SVM), Naïve Bayes and K-Nearest Neighbors (KNN) to classify a given URL into a phishing one or a legitimate one. Additionally, Scikit-learn offers functionality to evaluate models, to choose the best features and to perform a hyperparameter search.

We use the Pandas and NumPy modules for data processing and analysis. Pandas provides a simple high-level data structure for storing the data (a data series (1 dimensional, indexed, array of numbers or strings) and a data frame (2 dimensional table of numbers or strings)), as well as some basic methods for data cleaning. NumPy extends the basic array type provided by Python with additional features for efficient numerical computation. The data is then processed to a machine learning algorithm-ready format. This project uses Matplotlib for data visualization in order to create graphs and accuracy plots such as accuracy plot, confusion matrix, and model performance side by side. Visualization is crucial for this project in order to evaluate the performance of a model and communicate results. The backend of the system is implemented with web frameworks such as Flask and/or Django to handle server-side functionality and to route user requests to relevant points in the system to access external APIs as necessary. Flask is a lightweight application framework suitable for smaller systems and applications, and is often picked up and started writing code on very quickly. Django is a more full featured application framework that is geared towards large enterprise level systems and is generally easier to support and grow than other alternatives. The frontend of the system was developed using HTML, CSS, and JavaScript programming language. This is where users will type in provided URLs, and receive results. A simple and convenient system interface is very important for usability and user experience. Additionally the system incorporates external Application Programming Interfaces (APIs) to gather real-time threat intelligence, such as information about current malware and ransomware outbreaks from the Google Safe Browsing API. This external API is utilized to verify incoming URLs against a external database of known phishing websites, resulting in a higher detection rate of phishing emails compared to previously implemented approaches. The extracted features for each URL and their respective classification are stored in SQLite/MySQL databases. Efficient storage and retrieval of the data is critical for performance of the system. We use Git and GitHub to track the version control of our designs allowing us to track changes made and manage different versions of design code; also allowing us to work as a team. Phishman Defense combines machine learning, natural language processing and distributed computing power to provide an intelligent, scalable and effective anti-phishing solution.

Technique	**Description**	**Advantages**	**Limitations**
Blacklist-Based	Uses known phishing URLs	Simple and fast	Cannot detect new attacks
Rule-Based	Uses predefined rules	Easy to interpret	Limited flexibility
Machine Learning	Learns patterns from data	High accuracy	Requires training data
Ensemble Learning	Combines multiple models	Improved performance	More complex
API-Based Detection	Real-time verification	Updated data	Depends on external services

Table 2.1: Comparison of Phishing Detection Techniques

2.4 System View

This view provides a complete view of the proposed phishing detection system and describes how the individual components work together as a whole. It explains in more detail how the system operates and provides an overview of the flow of data through the system. The system is built using a number of individual components which have been organised in a modular fashion. Each individual component has a specific role to play in achieving the end goal of the system, which is to accurately and efficiently detect phishing web pages. The system follows a flow path from user input through to final classification classification and consists of all modular components that interact with each other seamlessly and with minimal time delay. The system is very scalable and is capable of processing real-time user requests. This is the start of the process in which the system analizes a given URL. The purpose of the Input Module is to receive the URL provided by the user, validate the information received by the system and check that it has been correctly provided in order to avoid any errors or anomalies that could appear during the analytical process. This Module also protects the system from threats by filtering out possible malicious inputs received from the user. This URL is then processed in the Data Preprocessing Stage where all the input data is cleaned and normalized. The preprocessing step can also include stripping of unwanted characters and normalisation of URLs as well as handling missing data. The goal of this step is to prepare the data to be used as training data for feature extraction. The Feature Extraction Module of our system plays a significant role in enabling URL classification. This component extracts a set of features from the input URL. These features can be mostly categorized into two types, namely lexical features and structural features. Some examples of lexical features are length of the URL, presence of special characters, number of sub domains and presence of suspicious keywords. Similarly, structural features can be thought of as features that describe the composition of the extracted features. Some examples include URL length feature, number of sub domains feature, age of domain feature, WHOIS records feature and DNS records feature. In analysis, feature extraction is required in order for machine learning to be applied as the model would have nothing to determine patterns and make predictions with otherwise. After processing all the features of the URLs, the program proceeds to the Machine Learning Module. This module uses various machine learning models and techniques to classify the provided URLs. It uses algorithms like Random Forest, Support Vector Machine (SVM), Naive Bayes and K-Nearest Neighbors (KNN) to classify the features of a given URL whether the given URL is phishing or not. All the models are used to increase the accuracy of the model. The system includes an Ensemble Module that increases accuracy and reliability of the classification results. In ensemble learning, for a sample of input, a prediction is generated using the predictions of multiple machine learning models. The methods for combining predictions include majority voting (simple ensemble) and weighted voting (regularised ensemble). The Ensemble Module reduces the number of false positives and false negatives.

In addition to utilizing machine learning approaches, Email security also incorporates a Rule-Based Module (RBM) that identifies entities within e-mail messages that could potentially be considered suspicious or even malicious. The Rule-Based Module enhances the overall accuracy rate when complemented with machine learning approaches to identify obvious phishing e-mails. Rule-Based Module is effective at identifying obvious phishing e-mails that contain URLs with an abundance of special characters or include domain names and keywords that are clearly suspicious. Another important component of the system is the API Integration Module, which provides real-time threat intelligence. This module links our system to outside services like the Google Safe Browsing API. It checks URLs in real time by looking at the latest lists of known phishing sites. This means we can catch new threats as they come up, even if they weren't in our training data. By connecting to the API, we can better handle phishing attacks that are always changing and evolving. The last part of the system is the Output Module, where it shows the user what it found out. It clearly says if a website is trying to trick you or if it's safe. Along with the answer, it might also give you more details like how sure it is about its decision or why it made that choice. This helps you understand what's going on and makes you trust the system more. The system's layout also shows how information moves between different parts. It goes from one part to the next in a straight line, making sure everything runs smoothly and quickly. Each part gets information from the one before it, works on it, and then sends the result to the next part. This organized way of doing things helps the system work well and reduces delays. When we look at a system, it's also important to think about how its different parts interact with each other. Even though each module works on its own, they are all connected and work together to reach the main goal. For instance, the part of the system that uses machine learning needs the part that extracts features to work properly. Meanwhile, the part that combines multiple models needs the results from several different models to do its job. In the same way, the API part helps the machine learning and rule-based parts by checking things in real time. This interaction between modules is what makes the system work smoothly and effectively. The system is designed to support real-time processing, which is essential for practical applications. Users expect immediate feedback when interacting with websites. Therefore, the system is optimized to provide fast response times while maintaining high accuracy. The system is designed to handle a lot of users at the same time without slowing down. It can manage its resources well and uses special algorithms to stay fast, even when it's really busy. This means it can deal with many requests from different users simultaneously, without any major drop in performance. The system's ability to scale and perform well is important, as it needs to be able to handle a large number of users and requests without any issues. Security is another important consideration in the system design. The system ensures that user input is validated and processed securely to prevent misuse. This plugin implements error checking and validation for better security. The system view explains in detail how the phishing detection system works, and describes how the system processes data, how different components interact with each other. The system uses machine learning, ensemble learning, rule-based detection, and API verification methods for accurate detection of phishing web pages.

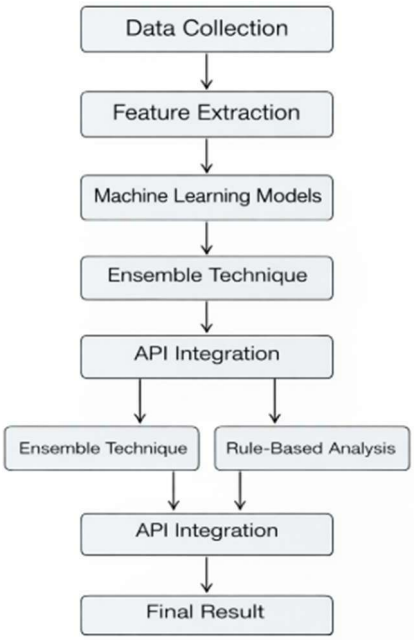

Figure 2.1: Methodology Workflow of Phishing Detection System
Source: Self-created (Napkin AI)

2.5 System Components & Functionalities

Our approach for phishing e-mail detection is based on a multi-modules setup. Each module serves a special purpose within the setup and all of them together enable an efficient and accurate detection of phishing e-mails. The setup is easy to maintain and scalable for large amounts of data because it is based on a multi-modules setup. The system starts with the Input Module. This is where the user enters a URL that he/she would like to analyze. The user enters a URL, the Input Module checks whether the entered URL is correct and the system starts the analysis process by passing the entered valid URL to rest of the system. The input is first read in and then passed to the Feature Extraction Module. This is one of the core parts of the system where the URL is transformed into features that can be classified by a machine learning model. We use lexical features like the length of the URL, the presence of special characters, the amount of subdomains and certain "suspicious" keywords. We also include domain related features like the age of the domain and the information provided in the WHOIS records. The quality of the features has a huge impact on the accuracy of the machine learning model. Extracted features are passed on to the Machine Learning Module. In this module, the URLs are classified as either phishing or genuine. The Machine Learning Module uses machine learning

algorithms to train and test the models. Random Forest, Support Vector Machine (SVM), Naive Bayes and K-Nearest Neighbors (KNN) are the models used in this system where four models are used to classify the URLs. To enhance accuracy of the predictions made by the machine learning models incorporated in the software, an Ensemble Module has been introduced in the current version of the software. Error reduction is a key benefit of Ensemble learning, utilising the strengths of individual models. Techniques for combining the output of multiple learning models include majority voting, where the most frequently returned class is chosen as output, and weighted voting, where weights are assigned to each model and the class with the highest total weight returned. Generally, Ensemble learning techniques improve system robustness and reduce error rate for classification type problems. The system implemented in the paper includes a Rule-Based Module that identifies obvious phishing emails and a machine learning based Spam classifier that identifies emails that may contain phishing content. Examples of rules for identifying phishing URLs include the presence of special characters in the URL, atypical domain structure, and presence of spam keywords in the URL. In addition to the prior features, the API Integration Module performs real time validation by connecting to external threat intelligence services such as Google Safe Browsing API to identify if provided URL's have been reported by known phishing databases, including recently discovered threats. The third and final component of the system is the Output Module that provides an interface to human users to display their analysed URLs with flag indicating whether the given URL is a phishing site or not; along with the output the system also provides confidence level or reasons for the decision made by the system. Components throughout the system collaborate to provide accurate, efficient, and timely phishing email detection.

2.6 Data & Relational Views

This section of the documentation covers the data and relational view of the phishing detection system. This view presents an overview of the data, covering what is collected, stored, processed and managed by the system. Efficient data management is crucial for the system to provide accurate predictions in a timely manner, and to achieve scalability. The approach presented in the paper relies mostly on large datasets of labeled phishing and normal websites (urls). These datasets are used to train Machine Learning algorithms where each url is classified as phishing or safe. We use our feature extraction module during the processing step to transform any given URL into a set of extracted features and arrange them into a structured data format, such as a table where each URL is a separate row and the features extracted are the new columns. Relational View of the system that outlines the relationships between entities in the system. The major entities involved in this view are: Objects, Classes, Frames, Frameset, Card, Container, Media and Regions.

• Experience Users experience while using the product; starting from initial setup, through the actions they perform while using the product.

• Url - user entered URL.

• endsWith: This method returns true if the URL ends with a certain string. This is useful for determining if a URL fits within a certain file type.

• Detection Result - Classification result (phishing or legitimate).

All of these objects are interconnected - a user can submit multiple URLs, each URL is associated with multiple features, and for each URL we generate detection results that the user can query the system for. We can store the URLs, features and results of the images in a database such as SQLite or MySQL. Our system can efficiently manage this database to look up the required data as fast as possible. The system can then scale up to millions of images. Proper indexing of the data is also important for optimal performance. Another aspect to consider in the data view is data preprocessing. This includes data cleaning, removing duplicates, handling missing values and normalizing the data. High quality input data is crucial for improving the accuracy and robustness of the models developed. Data security and privacy will also need to be addressed as the system will be processing user provided data. Correct validation and storage of this data is paramount to ensure users information is not compromised. We present a data and relational view-based approach to address this problem, enabling the system to work with data, establish relationships between various entities and perform accurate and scalable phishing detection.

CHAPTER 3:

PHISHING DETECTION SYSTEM: DESIGN CRITERIA

CHAPTER 3:

PHISHING DETECTION SYSTEM: DESIGN CRITERIA

3.1 Introduction

While building an accurate model for phishing detection is one of the key issues in building a phishing detection system, designing a system that can process large amounts of data and provide accurate results in real time is another critical issue. In this chapter, we discuss the design criteria of our proposed phishing detection system. We explain how different components of the system are designed, integrated and improved. The implemented system is based on a modular and layered architecture, wherein the system is composed of independent and several distributed modules, each one carrying out one or several tasks. These tasks are image capture, illumination normalization, feature extraction, image classification, validation and generation of results. The principal advantage of the proposed approach is that it is straightforward to understand and to maintain. In fact, it is even possible to update or optimize one of the existing functions independently of the rest of the system. In the field of cybersecurity, it is crucial to update the systems to deal with current threats. We built an extremely efficient system that has very well structured data flows. Starting from the user input, the workflow goes through preprocessing, feature extraction, machine learning classification, ensemble, rules and a test in the API.

- The overall workflow can be summarized as:

• URL input and validation

• Feature extraction from the URL

• Classification using machine learning models

• Ensemble-based decision making

• Rule-based and API validation

• Final result generation

Here's a rewritten version of the input text in a more human-like tone, similar to the provided reference human samples: When it comes to processing data, it's crucial to have a sequential flow in place. This ensures that everything is handled in an organized manner, which in turn minimizes the risk of errors and delays. Now, when we're talking about detecting phishing attempts, we need a system that's really good at catching the bad guys. To achieve this, the system uses a combination of different methods. By doing so, it becomes more accurate than if it were to rely on just one approach. The system utilizes machine learning to identify patterns in the data, which is a powerful tool in itself. But that's not all - it also combines predictions from multiple models to get a more comprehensive view of potential threats. Additionally, it checks for obvious threats using predefined rules, and verifies information in real-time using specialized APIs that are always up-to-date on the latest threats. By taking a multi-faceted approach, the system is able to detect phishing attempts with a high degree of accuracy, keeping people safe from those who would try to trick them. It's almost like having a team of experts working together to protect you from harm. Each component of the system plays a vital role, and together, they provide a robust defense against phishing attempts. This approach is not only effective but also necessary in today's digital landscape, where threats are constantly evolving. By leveraging the power of machine learning, combining predictions, and using real-time verification, we can stay one step ahead of the bad guys and keep our online interactions safe and secure.

- The combined approach provides:

• Better accuracy compared to single models

• Reduced false positives and false negatives

• Improved detection of unknown phishing attacks

In addition to the simple interface and security features, great scalability has also been implemented so that the system can handle many connections simultaneously and maintain its performance.

- This ensures that the system:

• Can support multiple users

• Maintains performance under load

• Can be expanded in the future

Security: The system secures the user input and processes it correctly. Also, the system handles errors correctly and validates the user input to prevent misuse.

For even greater flexibility, the system can be updated to include new machine learning approaches, features, and APIs as they become available. In addition to speed, design considerations were also made around how the product would integrate with other tools and components such as interface design, core processing, machine

learning models, and external APIs. How will the product be used efficiently, integrating within itself and with external tools.

- The system integrates:

• Frontend (user interaction)

• Backend (processing logic)

• ML models (classification)

• APIs (real-time validation)

This design can be used to share information across sections.

Phish Mail strikes a good balance between detection hits and false positives.

• Accuracy

• Efficiency

• Scalability

• Security

• Usability

The system is capable of detecting website phishing based on a robust and reliable way by taking advantage of the system's features of modularity and layered structure as well as multiple detection approaches. Next, we will discuss system implementation and evaluation.

3.2 Background

Phishing detection methods have evolved to combat the ever-changing landscape of threats in the current cybersecurity landscape. What used to be obvious indicators of phishing emails, such as misspellings in sender emails, have become less reliable as the tactics of email thieves have grown in sophistication. Organizations are looking for new and improved ways to prevent phishing emails from ever reaching employees' inboxes. Phishing detection systems are typically based on a blacklist approach where known phishing URLs are added to a database and then blocked by the system. However, as phishing tactics evolve, it is becoming increasingly more difficult for organizations to keep up with the constantly changing landscape of threats. To overcome these limitations, rule-based detection systems Systems to defend against URLs were developed as a complement to anti-virus software. Rule-based systems scans for suspicious URLs by looking for specific characteristics such as unusual domain names, presence of special characters and deception in keywords. Although they are more flexible than blacklists, they have another inherent problems in that they cannot keep up with the latest attacks that do not follow the rules.

Phishing detection using machine learning techniques to automatically identify phishing web pages has drawn considerable attention in recent years. Machine learning models, trained on large data sets, can be used to classify web pages into safe or phishing web pages. These models generalise to new unseen web pages, and can therefore enhance the effectiveness of traditional signature-based approaches that are incapable of dealing with newly generated phishing web pages. Recent studies show that the use of machine learning methods, including Random Forest and Support Vector Machine, can achieve higher accuracy levels in phishing web page detection than conventional methods.

- Some commonly used machine learning models include:

• Random Forest (RF)

• Support Vector Machine (SVM)

• Naive Bayes (NB)

• K-Nearest Neighbors (KNN)

In order to improve the accuracy of phishing detection, many methods have been put forward, which mainly utilize two learning paradigms, ensemble learning and traditional learning methods. For the ensemble learning method, Random Forest based on bootstrapping and decision tree can handle the large amount of training samples and prevent overfitting effectively. For the traditional learning methods, Support Vector Machine can handle the high dimension of feature space and find the global optimum decision boundary hyperplane. Therefore, many studies have adopted the two methods for phishing detection. Although great strides have been made recently in individual areas of machine learning, no current method can guarantee success on a particular problem and particular dataset. However, by training many models on a single training set and combining their results, we can harness the power of multiple models to improve performance. We discuss here how to train an ensemble of neural networks on a given dataset.

- Key benefits of ensemble learning include:

• Improved accuracy

• Better stability across datasets

• Reduced model-specific errors

Several methods such as majority voting and weighted voting/averaging have been adopted in the literature for phishing prediction. Using ensemble learning methods can improve the accuracy, robustness and trustworthiness of phishing detection systems.

Phishing detection has evolved to where Phishing detection now includes the detection of phishing e-mails in real-time. It even uses real-time threat intelligence to detect out malicious websites. Google Safe Browsing, an API based system, provides real time information on phishing URLs that have already been recognized.

- The advantage with such a system is that:

- It will be able to detect new emerging phishing URLs.
- The URL will also be regularly checked against the database to ensure that it is authentic.
- Such a strategy will aid in enhancing the detection accuracy.

Besides the technological development, there is also a great development in phishing attacks over the recent past. The current phishing tricks are HTTPS spoofing, URL shortening, URL redirection and heavy application of social engineering tricks. These changes indicate that there is a need to come up with a more sophisticated method of identifying such attacks. To achieve this one has to employ the developments made to date. The proposed system is based on an intelligent methodology in which a number of detection methods are combined. Patterns are detected with the help of machine learning, and the prediction process is made more accurate with the help of ensemble learning. Moreover, certain rule-based analysis could be employed to identify blatant threats as well. Lastly, real-time data may be obtained with the help of APIs.

- This strategy has a number of advantages, and they are:

• The detection of all types of phishing attacks.

• Minimization of false positive and negative results.

• More reliable and faster results.

The system will also possess a number of strengths over other detection methods that are in use, making it superior. It provides an integration of a number of detection methods, which means that it can deal with the vulnerability of other systems. In terms of research, the present project is correlated with the recent discoveries in cybersecurity. The importance of using a combination of multiple methods of detection in order to gain improved accuracy has been highlighted in many studies. Finally, the history of phishing detection is very clear that it is shifting to the less sophisticated towards intelligent methods.

3.3 System Design

System design explains how the different components of the phishing detection system can be combined into a system and how they interact. The design involves a logical series of operations which begin with the entry being made by the user and all the way to the result of the last

classification. The user interface sees the user input the URL in question. The information entered is then sent to the back-end system to process the information. On the back-end system, the input data undergoes such process as feature extraction which will be subsequently fed to the machine learning algorithms. The extracted features are used to make predictions using the models by the machine learning algorithms. The ensemble model is then used to combine these predictions to produce an ultimate output of the classification results. Moreover, the system also experiences rule-based validation and API verification to achieve improved results. The ultimate classification output is then sent back to the user interface where the end-user can view the result. The system allows free flow of information amongst modules and minimizes any latency in the process.

3.4 Design Diagrams (DFD, Use Case, Sequence)

Many design diagrams are employed in order to have a clear understanding of how the system operates and its structure. Diagrams can be applied in describing the flow of data within the system, interactions between different elements of the system and the flow of operations within the system. Instead of explaining it using mere words, diagrams are useful in simplifying the process. Design diagrams have a significant role to play in system design because they facilitate the simplification of system behavior, bottlenecks and interaction between different modules in the system. This particular project uses four major design diagrams: Data Flow Diagram (DFD Level 0), Data Flow Diagram (DFD Level 1), Use Case Diagram, and Sequence Diagram.

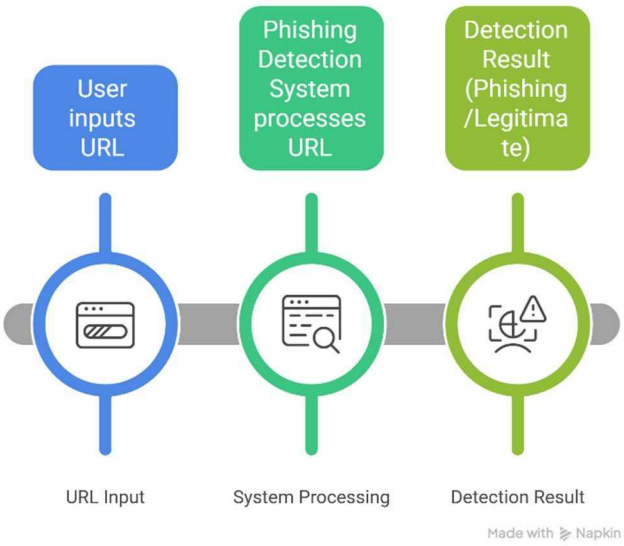

Figure 3.1: Data Flow Diagram (DFD Level 0)
Source: Self-created using Napkin.ai and other diagramming tools

The Level 0 Data Flow Diagram or the context diagram provides an idea about the phishing detection system in brief. It consists of a single process which is a manifestation of the whole system and any interactions between the entities outside of the system and the system itself are illustrated here. In this particular case, the entity external to the system is the user. The information given by the third party is a URL. After processing the information, the output of the system is the determination of whether it is a phishing site or not. The diagram does not give any detail of internal processes. The primary purpose of this diagram is to outline the extent of the system and provide a rough concept of the manner in which the system functions with respect to the external bodies.

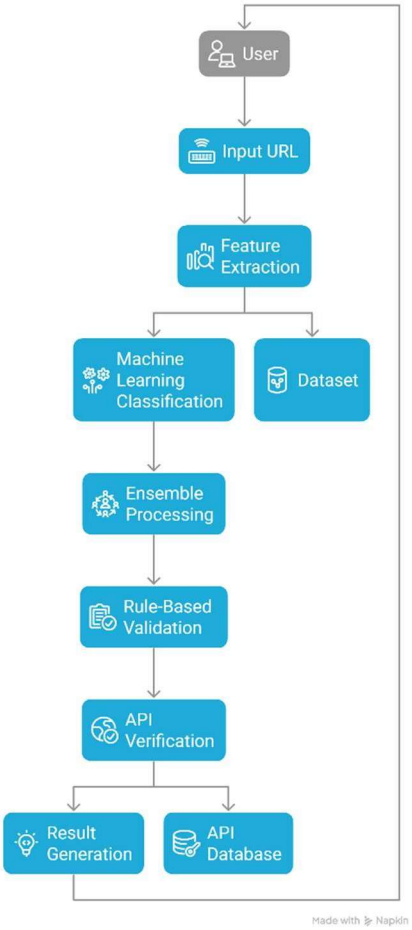

Figure 3.2: Data Flow Diagram (DFD Level 1)
Source: Self-created using Napkin.ai and other diagramming tools

The Level 0 Data Flow Diagram or the context diagram gives a brief idea of the phishing detection system. It contains a single process, which illustrates the whole system, and all the interactions between the entities outside of the system and the system itself are displayed here. Here, it is the user who is external to the system. The information given by the third party is a URL. Once the information has been processed, the system will produce an output that it is a phishing site or not. The diagram does not provide any details of internal processes.

The key role of this diagram is to specify the range of the system and provide a preliminary conception of how it works in the context of external objects.

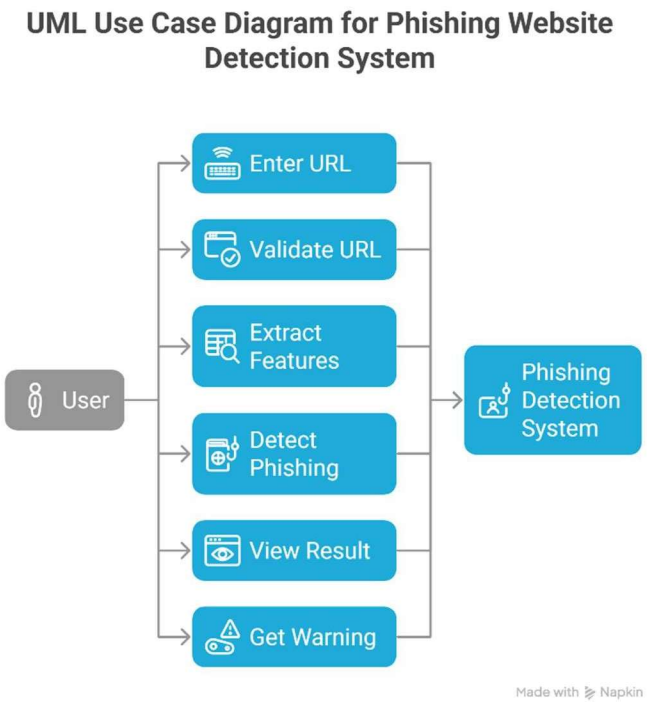

Figure 3.3: Use Case Diagram
Source: Self-created using Napkin.ai and other diagramming tools

The use case diagram explains a relationship between a user and the phishing detection system. It mainly deals with what the user can do through the functionalities provided by the system. As the primary actor, the user can perform such functionalities as typing a URL, verifying the URL, initiating phishing detection, obtaining results and warning prompts, which are required to be undertaken during the interaction. These functionalities are described in the use case diagram. It is important as it helps to identify the requirements and expectations of the user of the system.

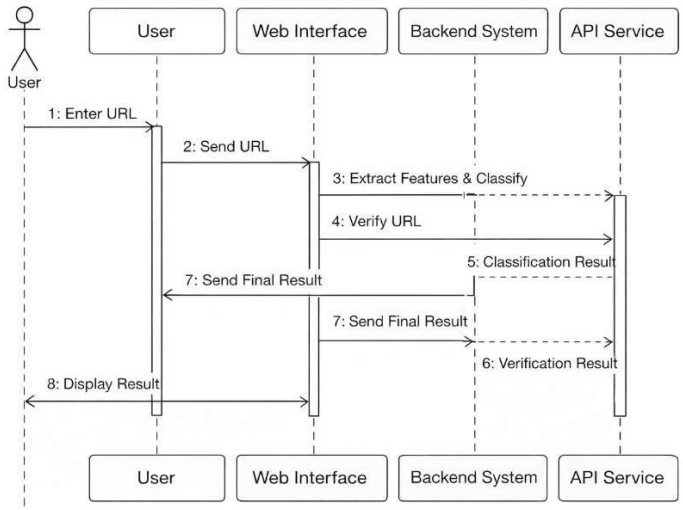

Figure 3.4: Sequence Diagram
Source: Self-created using Napkin.ai and other diagramming tools

The sequence diagram illustrates the interaction of the different parts of the system in time order. This is the diagram of the communication process between the user, web interface, backend and API services. First of all, it begins with the user typing the URL in the web interface. To that end, the web interface forwards the URL to the backend where features are mined and categorized. Then, the backend connects to an external API and validates the URL in relation to any phishing databases. When API responds, the backend will collect all the information and develop the ultimate classification. The next step is to transfer the result to the web interface where the user is provided with the information. This diagram is particularly useful in determining how the system behaves.

Requirement	Description	Priority	Module
URL Input	User can enter URL for analysis	High	Input Module

Requirement	Description	Priority	Module
Data Validation	Validate URL format and correctness	High	Preprocessing
Feature Extraction	Extract relevant URL features	High	Feature Module
Classification	Classify URL using ML models	High	ML Module
Ensemble Processing	Combine predictions from models	High	Ensemble Module
Rule-Based Detection	Detect obvious phishing patterns	Medium	Rule Module
API Integration	Verify URL using external APIs	High	API Module
Result Display	Show result to user clearly	High	Output Module

Table 3.1: Functional Requirements of the System

Requirement	Description	Importance
Accuracy	System must provide high detection accuracy	Critical
Performance	Fast response time (<2 sec)	High
Scalability	Handle multiple users simultaneously	High
Security	Protect user input and system data	Critical
Usability	Simple and user-friendly interface	High
Reliability	Consistent results across inputs	High
Maintainability	Easy to update and modify system	Medium

Table 3.2: Non-Functional Requirements of the System

3.5 System Architecture

The system architecture demonstrates the way in which the various elements of the anti-phishing system connect with each other and integrate. Good system architecture will enable you to process data quickly and easily.

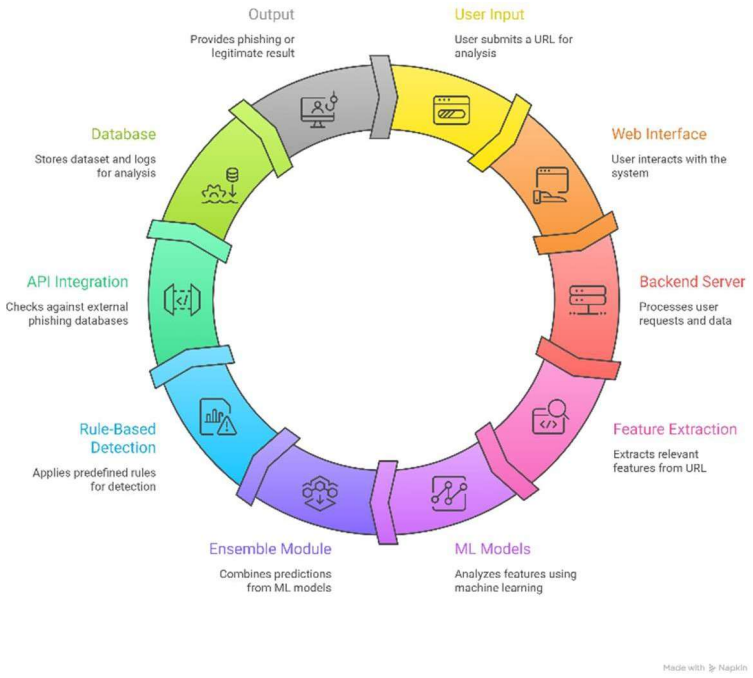

Figure 3.5: System Architecture Diagram
Source: Self-created using Napkin.ai and other diagramming tools

The system architecture is founded on the notion of layers. The layers perform a specific functionality and the combination of the layers have a complete system. The highest level is where a user interacts with the system by typing a URL. Within the framework of the current system, the frontend layer is the layer in which one interacts with the system. The frontend makes use of the HTTP protocol for communication with the backend. Inputted data were processed by backend processes which included a number of operations, such as feature extraction, classification and decision-making. ML algorithms are used at the backend layer and execute various models to work with data. The system has an extra layer of API that serves as a connector between the system and external services to permit real-time verification. In such a way, the architecture of the system is flexible, maintainable, and upgradable due to the layered architecture.

3.6 Information & Communication Design

This facet focuses on the process of data transmission, processing and handling within the system. Effective exchange of information amongst the system components is required to achieve fast and effective outcomes. There is some structure to data presentation. As an example, the URLs submitted are processed, and the output is in the form of feature vectors on the basis of which the data is classified by machine learning algorithms. Feature vectors themselves are presented in a certain structure. The frontend and backend are able to communicate through HTTP and/or HTTPS. Once a user enters a URL it will be processed and the results will be sent back to the user. Proper error handling is included in the system. An example would be invalid user input, which is identified and processed appropriately; errors arising when contacting third-party APIs are also appropriately handled to prevent crashes in the system and the like. The consideration should also be made on user interaction. In particular, users should be able to understand whether submitted URL is safe or potentially malicious by analyzing results of their submission to the system.

CHAPTER 4:

DEVELOPMENT & IMPLEMENTATION OF THE SYSTEM

CHAPTER 4:

DEVELOPMENT & IMPLEMENTATION OF THE SYSTEM

4.1 Introduction

This chapter presents the development and implementation aspects of the proposed phishing
This document describes the detection system. It provides a detailed explanation of how the system is practically built based on the theoretical basis.this test plan in the Capstone Project Part C: Test & Debugging chapter, which builds upon the design phase where the actual design of the microcontroller system is done. Ideas this chapter shows how we put them into practice. Capable of detecting phishing websites in real-world scenarios. Based on our experience of previous systems we have put together a guidance note for the implementation of a number of combined technologies. Machine learning tools, web application platforms and real-time Application Programming Interface (API) services. The technologies combined in this system provide not only accuracy, but also great efficiency and user friendliness. The phase "friendly." is aimed at a practical realization of the proposed design. The primary objective is to create a scaled down prototype. A program designed to handle user input, analyze websites and produce correct results. The development process is systematic and structured component is implemented correctly. It begins with data collection, where datasets containing. The negative URLs (both phishing and legitimate) are collected and then preprocessed. The data is then cleaned and preprocessed. It is then subjected to feature extraction, which turns the raw URLs into features. Chemicals have relevant properties that can serve as meaningful attributes for machine learning models. Machine learning models are then trained with the extracted features. Phishing detection models learn features which help to differentiate between phishing and benign websites. After training the models, they can be integrated into the existing system in order to classify online URLS in real time. real-time capabilities. All forecasts are continuously validated against established weather forecasting rules and are combined into ensembles to optimize predictive accuracy. The system is accessible through a dedicated API integration to enhance detection accuracy. The workflow is to generate a list of images to render, run render jobs through rendertron for each one, then put the rendered frames in a directory for the next step to process.

- Collection of phishing and legitimate URL datasets

- Data preprocessing and cleaning
- Feature extraction from URLs
- Training of machine learning models
- Integration of models into the system
- Implementation of user interface and API services

The processes are clearly defined to allow the system to function correctly. System integration plays an important role in the implementation. Various components like web servers, load balancers, proxy servers and the like have to be properly configured to achieve a robust system that scales to higher user counts and operates around the clock other essential components and their interplay in the platform. We have external APIs that need to integrate with each other, and with appropriate integration, data can flow correctly minimizes delays in processing. This software is also optimized for performance and ease of use. Real world applications have been developed using this software to test and verify its capabilities. This command must complete instantly so that the user is not kept waiting for its results optimized to provide fast results without compromising accuracy. Each step is carefully designed and executed in order to make sure that the system operates as efficiently as possible while providing accurate outputs. System integration is another essential component of the implementation process. Components including frontend, backend processing system, machine learning model, and external APIs have to be integrated correctly in order to ensure efficient data exchange and avoid any possible delays while working with data. Finally, special attention should be paid to performance and usability of the system. When developing a tool which would be used in a practical application, the user expects immediate results of interaction with the program, hence it is important that the implementation ensures high performance while being user-friendly.

- Among the main goals of the system there are several important aspects which are considered at each step of implementation:

- Real-time detection of phishing websites
- High precision classification results
- Fast processing speed
- User-friendly interface

In particular, user experience is ensured through creation of an easy-to-use interface for interaction with the program. All that the user has to do in order to get the required information about a certain URL is to enter the URL and get a ready result a whether this website is safe or not.Moreover, special attention is paid to safety during implementation. Validation of user input data and error handling mechanism are included into the process of system implementation in order to minimize risks of any possible issues related to security. As a final point, scalability is considered at each stage of development, and the system should be scalable enough to be able to serve a number of users and potentially increase its scope in the future.

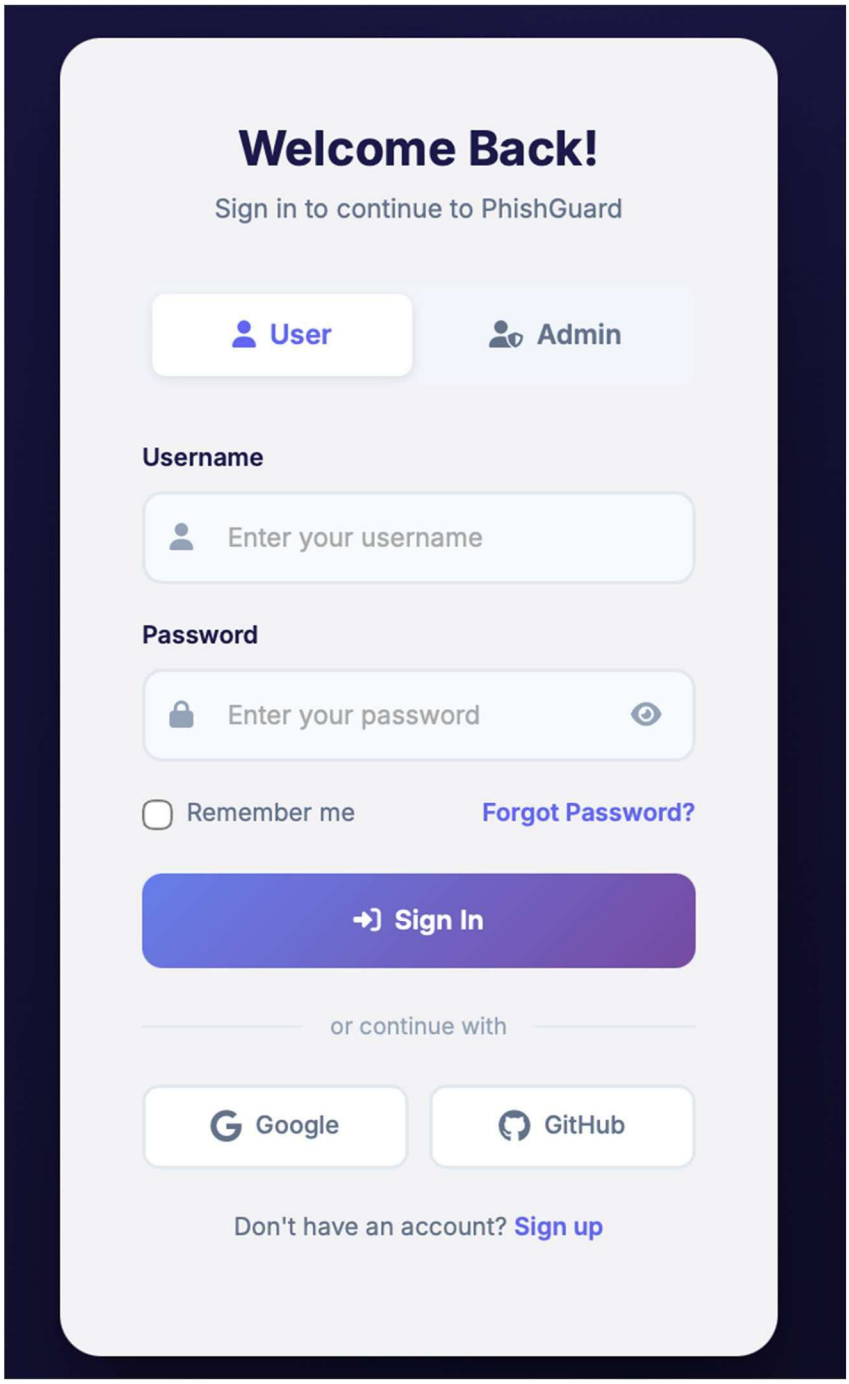

Figure 4.1: Login Page
Source: Self-created

4.2 Related Work (Literature Review)

Detection of phishing attacks has received attention in the domain of cybersecurity because of the rapid growth of online frauds and cyber-attacks. In recent times, several methods have been introduced that focus on detecting and preventing phishing attacks through phishing websites. However, these methods have evolved through different eras of technologies such as rule-based, machine learning, and hybrid models. Therefore, knowledge about the available solutions will help in identifying the drawbacks of each approach and developing a better approach. Initially, phishing detection approaches used the concept of blacklisting where a list of all the detected phishing websites was added to the database and prevented. Such an approach was very straightforward but it had one disadvantage: only those sites were detected which were already present in the database. Due to the constant creation of new URLs by attackers, these methods proved insufficient in detecting new attacks. To this end, rules-based systems were developed as a solution to the limitation that blacklisting posed. As a solution to the limitation of blacklisting, was the introduction of rules-based systems whose rules would identify any unusual pattern in a URL. Such patterns include suspicious domain name, presence of special characters in the URL, and keywords likely to deceive users. Although the rules-based system was efficient in alleviating the inflexibility problem of blacklists, the approach could not accommodate the present phishing trends. Therefore, the rules-based algorithms managed to detect all new cases of phishing patterns that are within its rules. Technological progress has led to the development by researchers of a powerful method of phishing site identification. Therefore, there is a possibility of developing predictive models for predicting phishing in any website based on machine learning models. Machine learning models are unique from other traditional models because they are able to predict any phishing website that is not part of the training datasets. Hence, machine learning models are best suited for application in reality. According to researches [3] and [9], machine learning models have been proven to predict phishing websites accurately. Random Forest and Support Vector Machine (SVM) machine learning algorithms can effectively classify URLs based on URL features. The performance of Random Forest is high compared to the SVM since it can resist overfitting due to the random decision tree. On the other hand, SVM can handle multidimensional data.

- There are several machine learning algorithms which could be useful in detecting phishes as follows:

- Random Forest (RF)

- Support Vector Machine (SVM)
- Naive Bayes (NB)
- K-Nearest Neighbor (KNN)

Although there are some advantages with the use of the above-mentioned machine learning algorithms, there are several drawbacks that need to be addressed. For example, a particular algorithm may work well in one data set, but it will not necessarily be effective with another data set. Moreover, while some of these algorithms tend to have overfitting problems, others are unable to recognize complicated patterns. To overcome the mentioned issues, scientists came up with ensemble learning. One more innovation that is to be mentioned is a usage of real-time APIs to identify the phishing URLs. The contemporary phishing detection mechanisms tend to rely on Google Safe Browsing APIs and compare the URL of the website with a list of phishing websites. The most important thing is that with the help of such an approach, it is possible to identify phishing pages even when it is not present in the training set. APIs are highly used, which enhances system reliability. In this way, extra level of verification is provided. The main advantages of using APIs are

- On-the-fly threat detection.
- Checking the URLs against an up-to-date base of phishing sites
- Higher reliability

Complexity and diversity of methods employed by attackers is another innovation related to the modern day phishing attacks. The key point is, hTTPS spoofing, URL obfuscation, redirection – all these tricks are utilized by attackers to bypass the detection system. so, a more advanced approach must be developed. However, the most important thing to consider is that a hybrid approach, adopted by the proposed system, can be used to achieve a combination of the benefits of various detection methods. The hybrid method will mean that the machine learning methods will be used in combination with the ensemble learning and a rule-based analysis and the API check. The important thing is, this system is more accurate and reliable due to:

- Combination of different detection techniques
- Ensemble learning to boost the accuracy of classification.
- Performing real-time validation via APIs

This kind of hybrid solution is quite typical of the contemporary cybersecurity solutions because of its advantages. As far as research is concerned, a hybrid system gives far superior results compared to a system that uses one technique only

4.3 Developmental Feasibility

The assessment of feasibility of any system must be carried out before any system is implemented to make sure that the proposed solution is capable of being successfully developed, deployed and maintained in a real life situation. Feasibility analysis assists in determining the possible challenges, resources needed and the general practicability of the system. The proposed phishing detection system has been tested in various aspects, such as technical, operational, and economic aspects, and other feasibility factors.

Technical Feasibility

Technical feasibility is the ability to develop the system utilising the existing technology and resources. The phishing detection system presented is technically viable because it is created based on technologies that are used and supported by many. A Python language is used to develop the system, and it is one of the most popular programming languages to work with machine learning and data analysis. It offers comprehensive libraries like Scikit-learn, Pandas, and NumPy, which make it easy to implement machine learning models and data processing tasks. The libraries are credible, effective and have a wide range of applications in academics and industries. Moreover, the backend of the system is developed using such web frameworks as Flask or Django. These architectures will facilitate API integration and enable easy interaction between various system components. The frontend is built with regular web technologies like HTML, CSS and JavaScript, making it compatible with various platforms. System integration is another crucial factor of technical feasibility. The suggested system is a combination of several elements such as machine learning models, rule-based analysis, and external APIs. In spite of this complexity the integration is possible since these components are compatible and able to communicate effectively with standard protocols.

- The system is technically viable in the sense that:

  - It operates programming languages and tools that are readily available.
  - There are machine learning libraries that facilitate effective implementation of models.
  - Web frameworks facilitate easy development of back-ends.
  - Standard protocols are used in API integration.
  - In general, the system can be created with the help of the technologies that are already available and do not need any specific or unattainable resources.

Operational Feasibility

Another important factor is system reliability. The combination of machine learning models, ensemble methods and API verification is used to make sure that the system gives correct and consistent results. The system is also structured in such a way that it manages errors and ensures operational feasibility measures the effectiveness of the system to operate in the real-life environment and whether the system is feasible to the users. The system of phishing detection proposed is very possible to operate. The system will be user friendly and simple to use. In order to use the system, the users do not require any technical knowledge. All they have to do is to type in a URL and the system will give the answer whether the site is a phishing or a genuine site or not. This simplicity makes sure that the system can be operated by a large group of users including non-technical users. The outputs are also clear and understandable in the system, aiding users to make informed decisions. The findings are presented in a straightforward manner, which makes it less confusing and easier to use.

hat the system operates well even when there are unforeseen circumstances.W Operational feasibility is guaranteed because:

- Operational feasibility is ensured because:

- The system is user-friendly and needs the minimum user effort.
- The interface is easy to use and understand.
- The system has good and precise results.
- System stability is enhanced by error handling mechanisms.

Economic Feasibility

Economic feasibility analyses the cost-effectiveness of the system. The phishing detection system proposed is also cost effective because the system is built with the help of open-source tools and technologies. The system requires neither costly hardware or software. The development tools (Python and machine learning libraries) are free. This greatly lowers the general cost of development. Moreover, the system can be implemented on regular computing devices, without having to install specialized infrastructure. There are also low maintenance costs because it is possible to update and improve with the help of existing tools.

- The economic feasibility is realized due to the fact:

- The cost of development is lowered with open-source tools.
- There is no costly hardware needed.
- Maintenance and upgrade is at low costs.
- This renders the system suitable both academically and practically.
- This makes the system suitable for both academic and practical applications.

- **Scalability Feasibility (Additional Factor)**

Scalability is the capacity of the system to accommodate the growing workloads and users. This proposed system can be scaled because of its modular design and effective architecture.

The system will be capable of managing a number of user requests at a time without a major performance hit. New functionality may also be added in the future, adding new functionality, models, or APIs.

- **Performance Feasibility (Additional Factor)**

Performance feasibility is an assurance that the system will adhere to the necessary speed and efficiency requirements. The system will work to give real-time results with minimal latency. Fast response time is important to phishing detection systems and is achieved through efficient algorithms and optimization of data processing techniques.

Security Feasibility (Additional Factor)

Since the system deals with user inputs and cybersecurity data, security is an important consideration. The system guarantees appropriate input checks, data management, as well as protection against abuse. This renders the system dependable and safe to be deployed in real world.

4.4 Implementation Specifications

The phishing detection system implementation includes integration of various technologies and tools, which collaborate to form an efficient and scalable solution. All the elements of the system are created with the help of appropriate technologies to achieve smooth performance, high-level performance, and reliability. The choice of technologies is very important in defining the overall effectiveness of the system. The implementation principles are founded on the core programming language Python, which is popular because of its simplicity, flexibility, and wide variety of machine learning and data analysis libraries. Python has many libraries and frameworks that simplify the use of complex algorithms to work with large data volumes. It is also simple to read and use, thereby accelerating the system development and maintenance. Machine learning models are implemented with the Scikit-learn library, which provides a complete set of classification, regression, and model evaluation tools. The library can be particularly used in the implementation of algorithms such as Random Forest, Support Vector Machine, Naive Bayes, and K-Nearest Neighbors. Scikit-learn also offers built-in capabilities for model training, testing, and evaluation, so it is less complex to write. The system manipulates and processes data using Pandas and NumPy. Pandas can be used to process structured data, clean data, and operate with tabular data. NumPy, however, is a package employed in numerical computations and is efficient in manipulating large arrays and matrices. Combined, these libraries make sure that data preprocessing and feature extraction are conducted efficiently. Matplotlib is also utilized in the system for visualization. Visualization plays an important role in the analysis of model performance and cognition of the tendencies in the data. The graphs generated to evaluate the performance of different models include accuracy plots, confusion matrices, and performance comparisons.

The backend of the system is developed with web frameworks (Flask or Django). These frameworks deal with server-side logic, such as user input processing, machine learning model execution, and API communication. Flask is small and can be utilized in smaller applications, whereas Django has more structured code for larger applications. The backend will ensure integration of all the system components and will ensure that the system operates smoothly. The frontend is developed using the common web technologies HTML, CSS, and JavaScript. These technologies are utilized to design a user-friendly interface such that the users are able to interact with the system easily. The interface is user-friendly and easy to use, enabling one to key in URLs and view results without necessarily being technologically minded. The other important aspect of the implementation is API integration. The system uses external APIs such as Google Safe Browsing as real-time checkers of URLs. This enhances the system to detect new phishing websites that may not be part of the training data. The key aspects of implementation of the system are:

- The key implementation components of the system include:

- Programming Language: Python
- Machine Learning Library: Scikit-learn.
- Numbers Processing: NumPy and Pandas.
- Visualization: Matplotlib
- Backend Framework: Flask or Django.
- Frontend Technologies: HTML, CSS, JavaScript.
- API Integration: Google safe browsing.

All these elements have a certain role in the system and can help improve the performance and functionality of the entire system.

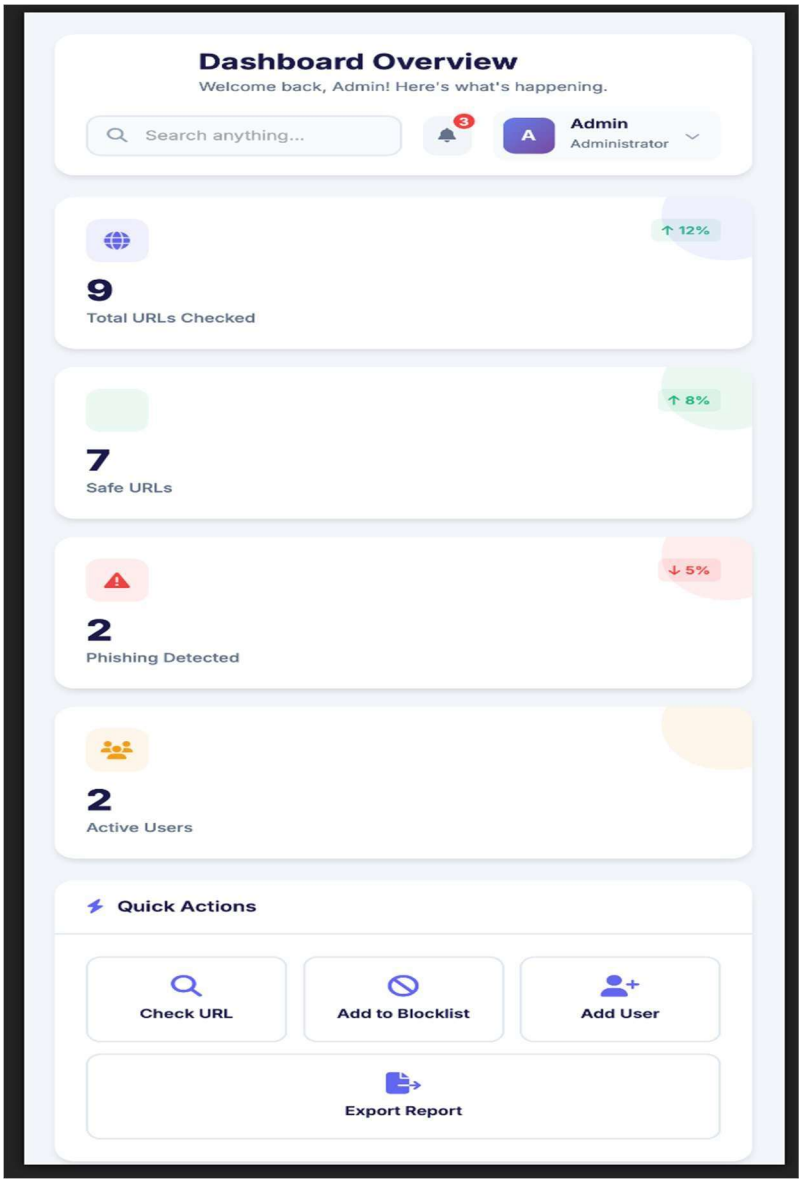

Figure 4.2: Dashboard
Source: Self-created

4.5 Training and Evaluation

The training and evaluation phase is one of the most critical stages in the development of the phishing detection system. In this phase, machine learning models are trained using labeled datasets and evaluated to determine their effectiveness in classifying phishing and legitimate URLs. It starts with data preprocessing, which includes cleaning and pre-processing of the data to be analyzed. Raw data can be inconsistent, missing values and irrelevant which can adversely impact model performance. Thus, preprocessing is done to provide quality and consistency of data. This step might involve eliminating duplicates, addressing missing values, and standardizing the URL format. Like preprocessing, feature extraction follows. Machine learning models do not accept raw URLs directly and hence they need to be transformed into numerical features. These characteristics are different attributes of the URL, including the URL length, presence of special characters, domain data and structural patterns. The extraction of features is an important process since the features are what determine the performance of the models. After the extraction of the features, the dataset is separated into training and testing sets. The training set is used to train the models, while the testing set is used to evaluate their performance. This division insures that the models are being tested on unknown data, which gives a more precise measure of their performance. The ready dataset is then fed into multiple machine learning models which are then trained. Such models are Random Forest, Support Vector machine, Naive Bayes and K-Nearest Neighbors. All the models possess their advantages and features. As an example, Random Forest can be used effectively in large datasets and decreasing overfitting, and SVM is recognized to be effective in creating optimal decision boundaries. When training, each model is taught patterns to differentiate between phishing and legitimate URLs. The models modify their parameters according to the training data to enhance the level of their predictions. Upon training, the models are tested with different measures of performance. These measures are quantitative indicators of the performance of the models.

- The most important assessment measurements are:

Overall accuracy of the model. Precision: shows the number of predicted phishing URLs that are really phishing. Recall: Measures ability to identify real phishing URLs. F1-Score: Gives a tradeoff between recall and precision. The metrics are useful to compare various models and choose the most appropriate ones to be used in the system. The process of evaluation will include the testing of every model on the testing dataset and the evaluation of the model. Models that prove to be effective are then chosen to be used further based on the results. The overall evaluation workflow includes: Preprocessing and cleaning of data. Feature extraction Dividing data into training and testing. Training machine learning models. Performance assessment of models with metrics. Comparison of models to choose the most appropriate models. In order to achieve additional accuracy, the system integrates an ensemble method, in which the predictions of a number of models are fused. This method minimizes errors and offers more consistent results than single models.

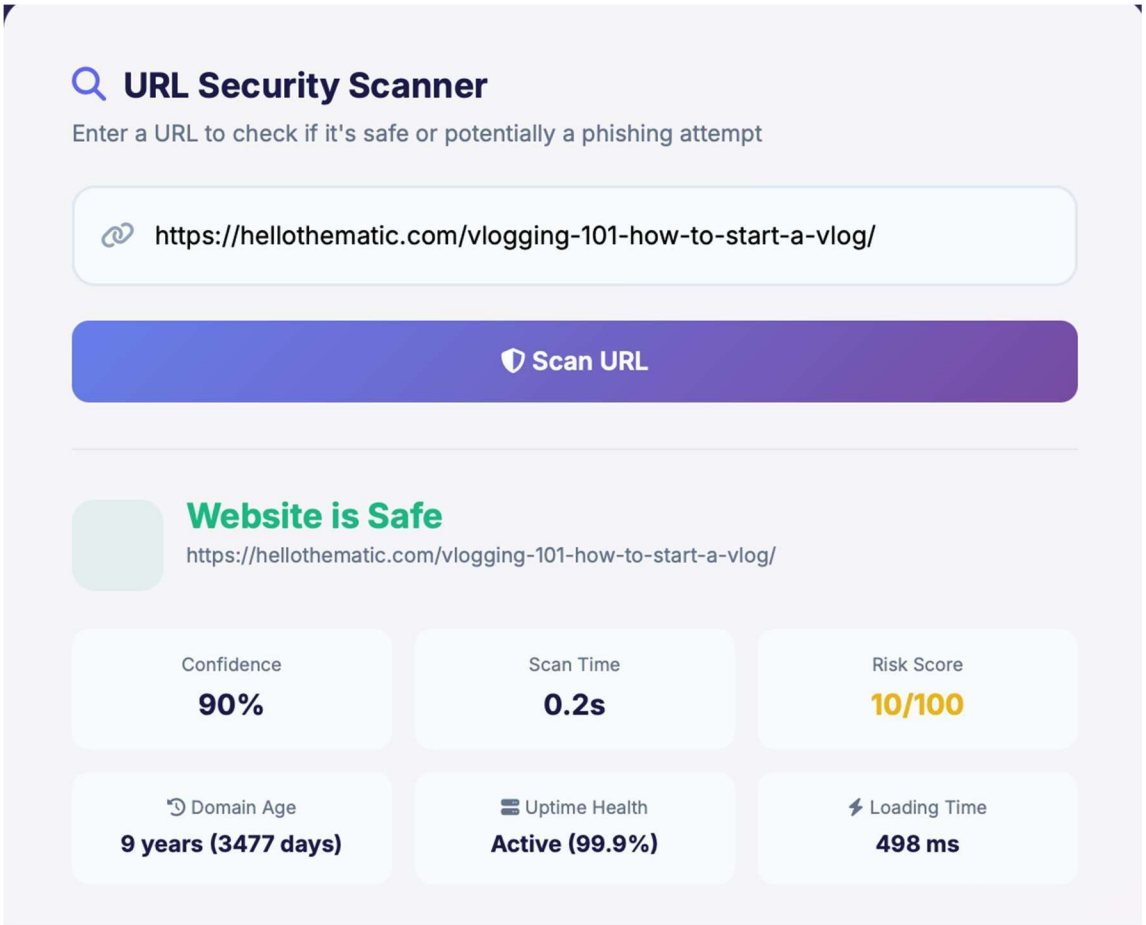

Figure 4.3: URL Detection Result Page
Source: Self-created

The multi-model approach will provide the system with the strength of each of the models without too much of the weaknesses of the models. This leads to better detection accuracy and reliability which makes the system effective in detecting phishing websites.

4.6 Critical Modules of the System

The phishing detection system has a number of critical modules which collaborate to carry out the detection process. The Input Module enables users to input URLs to analyze. The system provides validation of the input and proper formatting to be processed. The Feature Extraction Module examines the URL and extracts the features. These characteristics play a critical role in machine learning classification. The Machine Learning Module makes use of trained models to classify the URL. Multiple models are used to improve detection accuracy. The Ensemble Module is a combination of predictions of various models that eliminates errors and enhances reliability. The Rule-Based Module identifies blatant phishing rules, which offer an extra security measure. The

API Module will do real time check by comparing URL to current databases. Output Module represents the final output to the user in an understandable format.

➢ These modules make sure that the system is:

- Accurate
- Efficient
- Reliable
- Scalable

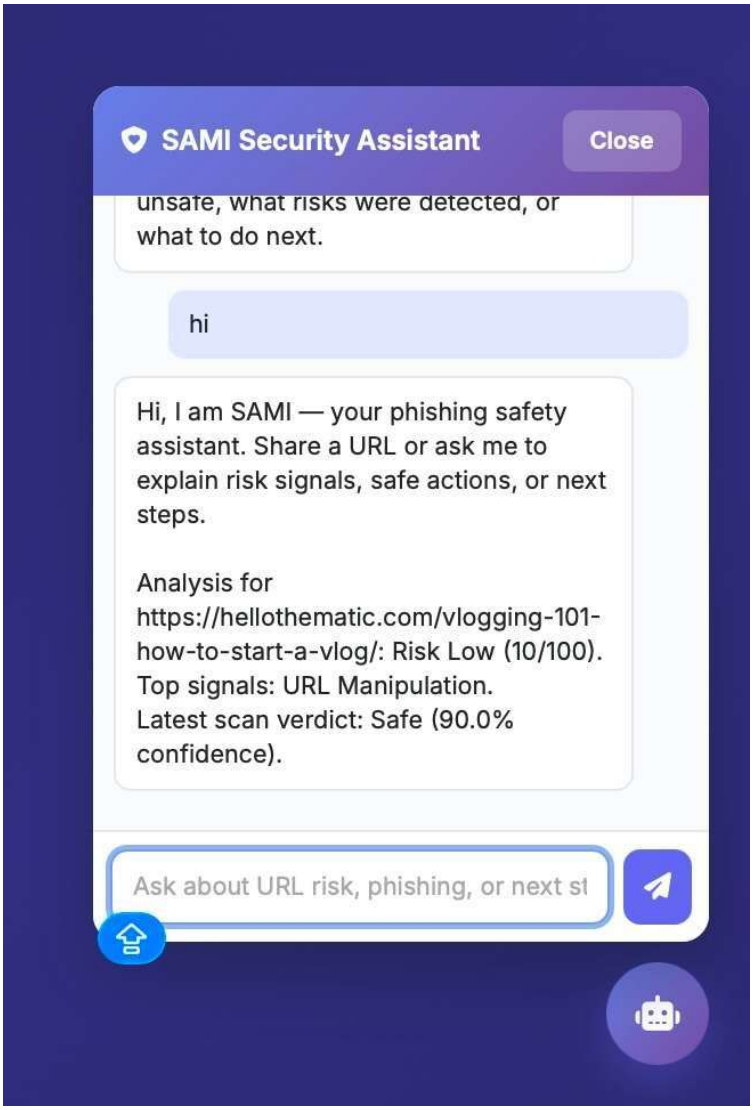

Figure 4.4: Chat-bot Interface
Source: Self-created

CHAPTER 5:

MACHINE LEARNING & ENSEMBLE-BASED PHISHING CLASSIFICATION

CHAPTER 5:

MACHINE LEARNING & ENSEMBLE-BASED PHISHING CLASSIFICATION

5.1 Introduction

Machine learning has become one of the most efficient and mighty in terms of solving intricate problems within the realm of cybersecurity, especially phishing-detection. Much with the increasing rate at which the internet and online services are being used, the phishing attacks have also advanced and become harder to be detected using traditional means. Previously used methods like blacklist-based and rule-based systems worked well to detect existing threats only, but they cannot detect new phishing sites that are created and modified dynamically. This leads to the high demand of smart systems able to learn and adapt with changing patterns of attacks. The machine learning in this case is a strong solution because it allows systems to process a large amount of data, detect latent patterns, and categorize URLs according to the learned features. Machine learning models can identify a previously unseen phishing website as opposed to using traditional methods, thus can be more applicable to real-world situations. These models take features derived out of URLs including structural patterns, domain information and lexical features to differentiate phishing and legitimate websites. The given chapter is dedicated to machine learning and ensemble approaches that will be used in the proposed phishing detection system. It describes the implementation of various classification models, their prediction integration and the overall system being highly accurate and reliable. The system is aimed at categorizing the URLs into two groups (phishing and legitimate) on the basis of the analysis of the extracted features. The system does not use a single model, instead it follows an ensemble-based approach which involves the use of several machine learning models. Both models process the input data separately and give a forecast. The results of these predictions are then put together to come up with a final result. The methodology can be used to minimize errors and enhances overall performance, as it builds upon the merits of various models and downplays their flaws.Ø The most important aims of this chapter are:

- The use of machine learning to detect phishing correctly.
- Boosting performance with ensemble learning techniques.
- Minimizing false negatives and false positives.
- Real-time classification in a fast and reliable manner.

The second significant benefit of machine learning is that it can be improved as time goes on. The models can be retrained and updated as the available data increases, to improve their performance.

Such flexibility provides machine learning-based systems with better performance in dealing with contemporary phishing attacks. In general, this chapter brings out the integration of machine learning and ensemble methods in the system to form a robust and efficient phishing solution. The system attains enhanced accuracy, improved generalization, and increased ability to identify known and unknown phishing threats by integrating several models and utilizing data-driven methods to improve its performance

5.2 Background

Phishing detection using machine learning is increasingly popular in the past years since cyberattacks are increasingly intricate and frequent. With the availability of additional and newer online platforms, attackers are ever devising methods of deceiving people and beating the normal security systems. This has made smart detection systems which are capable of keeping abreast with new threats a lot in demand. Machine learning is an appropriate solution since it allows system to analyse data, identify patterns and accurate predictions. The machine learning models take the input in form of a set of features extracted from URL. These are informative attributes of the URL that can be used to differentiate between phish sites and legitimate sites. The key distinction between machine learning models and traditional ones is that it automatically trains them with data, instead of being trained by pre-defined rules.

In phishing detection that is based on ML, feature engineering is crucial. These properties assist in deriving different aspects of URL that entail its structure, content, and security. Some of the features are:

- Length of URL.
- Presence of special characters.
- Domain data (age, registration information).
- Availability of HTTPS and SSL certificate.
- Suspicious keywords

These characteristics are transformed into numbers, and it is these numbers that are fed into machine learning models. These inputs are examined by the models and categorized URLs according to the patterns learned. A number of machine learning algorithms have been popularly applied in detecting phishing. The most common of them are Random Forest, Support Vector machine (SVM) and Naive Bayes as well as K-Nearest Neighbors (KNN). Random Forest is among the most efficient algorithms since it uses several decision trees to come up with a final prediction. This eliminates overfitting and enhances accuracy. Support Vector Machine is also characterized by the fact that it can process high-dimensional data and produce

the best decision boundaries. Naive Bayes is straightforward, quick and effective hence applicable to large data sets. KNN can be applied in pattern recognition, whereby it categorizes data in terms of similarity to other data points. In spite of the fact that these models work well when used separately, they all have some limitations. Certain models can be better on certain data sets and others can be unable to handle intricate designs. This contradiction brings out the necessity of a more vigorous approach. Ensemble learning methods are employed to overcome this shortcoming. Ensemble learning involves the use of a combination of machine learning models to enhance the overall results. The system is based on the combination of predictions of several models, rather than the one-point model.

➢ Ensemble learning offers a number of benefits:

- Enhances accuracy through strengths of other models.
- Increases stability among various datasets.
- Gives improved generalization to invisible data

Popular methods to combine ensembles are majority voting (where the ultimate prediction is made based on the most prevalent output of the models) and weighted averaging (where the models are weighted in accordance with their performance). Besides machine learning and ensemble techniques, contemporary phishing detection systems are usually combined with rule-based validation and API-based validation. Rule engines assist in identifying blatant phishing attacks very quickly and the APIs give real-time data regarding suspicious sites. This blend will result in a hybrid system which is more efficient than adopting one approach. The suggested system adheres to this hybrid style as it combines machine learning models, ensemble methods, rule-based analysis, and API verification. This is to make sure that the system is able to identify known phishing sites (using API and rules) and unknown threats (using machine learning). In general, the history of this system is the shift towards higher intelligent systems with the past traditional detection methods. The integration of various methods leads to a better accuracy, flexibility, and reliability, and the system should be used in the real-life scenarios of phishing detection.

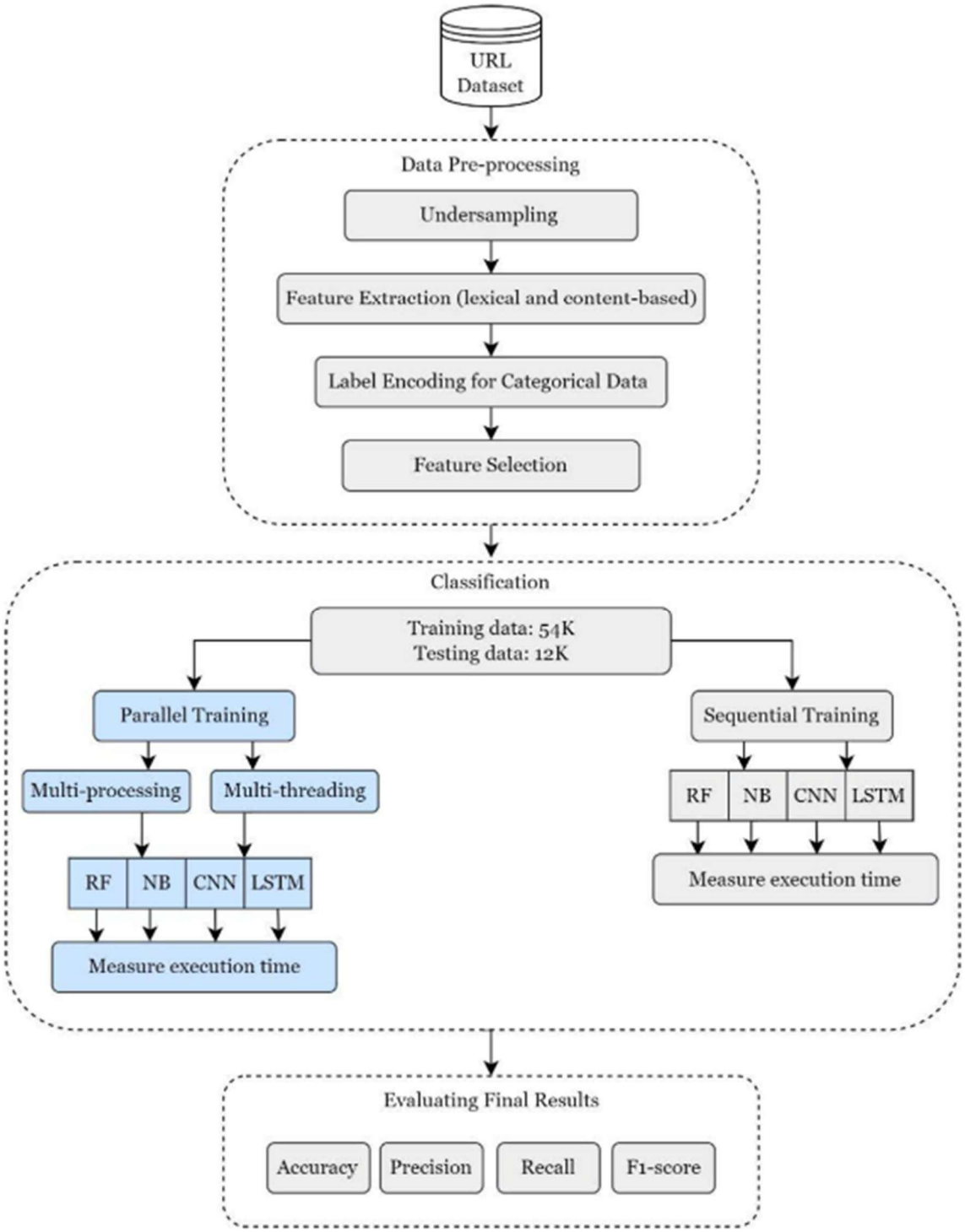

Figure 5.1: Methodology Workflow of the Proposed Phishing Detection System
Source: Self-created using Napkin.ai and other diagramming tools

5.3 Coding and Implementation

The phishing detection system coding and implementation has several processes that convert the raw data to a fully operational application that could identify phishing websites in real time.
This process involves data preprocessing, feature extraction, model training, evaluation and system integration. The stages are implemented with much care so that the system is efficient, accurately implemented and scalable. The system is written in Python, a popular programming language with machine learning and data processing applications, as it is simple and has a rich library ecosystem. Python allows creating an environment of flexibility in which data analysis, model building, and web integration can be done effectively in one framework. The first task in the implementation process is the loading of the dataset which is a collection of labeled URLs either phishing or legitimate. The data is normally saved as CSV and loaded through the Pandas library. After the dataset is loaded, it is preprocessed to clean and standardize the data. This involves deleting the missing values, removing duplicates and making sure that all the URLs are in a uniform format. The feature extraction is the next step after preprocessing, and it is one of the most significant elements of the system. Machine learning models cannot directly work with raw URLs, and thus they have to be transformed into numerical features. These attributes depict various attributes of the URL including its length, format, use of special characters, domain information, and security.Ø Characteristic features that are obtained in the system are:

- Typical features extracted in the system include:

- Length of the URL
- Number of special characters (@, -, _, etc.)
- Presence of HTTPS
- Domain age details and registration details.
- Keywords in the URL that are suspicious

These learned features are written in a structured format and are employed as feeds in machine learning models. After the preparation of the features, the system goes on to model training. Various machine learning models are being deployed with Scikit-learn. The models are Random Forest, Support Vector Machine (SVM), Naive Bayes and K-Nearest Neighbors (KNN). Training is done separately on each of the models using the training dataset. The models are trained to identify the patterns which differentiate between the phishing and legitimate URLs. This is done by modifying model parameters according to input data so as to reduce classification errors. Each model is tested using unknown data after being trained to assess the performance of the model. The second step is model evaluation where models are compared on the basis of performance factors like accuracy, precision, recall and F1-score. These measures can aid in identifying the effectiveness of each model at identifying phishing websites.

- The coding process could be summarized as
- Loading dataset using Pandas
- Preprocessing and cleaning of data.
- Parsing out features in URLs.
- Feedforward testing: Training many machine learning models.
- Evaluating model performance
- Selecting best-performing models

Following the analysis of single models, the system adopts ensemble learning, in which the prediction of several models are integrated. Majority voting is usually employed to do this. Ensemble learning enhances the accuracy of the overall system and minimizes the possibility of wrong predictions. The last phase of implementation will be system integration and in this case the trained models are linked to the back system. The back engine is developed with Flask or Django to address the requests made by the user and handle the input URLs. As a user types a URL, the system surrogate features, uses machine learning models, and physically devises a prediction. The interface is made simple, with the help of HTML, CSS, and JavaScript and offers the frontend to the users, through which they can interact with the system. External APIs are also incorporated into the system, including Google Safe Browsing to check URLs and increase the search power. This full implementation allows the system to be able to handle human input, URL analysis and give correct returns on a real time basis.

Component	Technology Used	Purpose
Programming Language	Python	Core development and ML implementation
Machine Learning Library	Scikit-learn	Model training and evaluation
Data Processing	Pandas, NumPy	Data cleaning and feature handling
Visualization	Matplotlib	Performance analysis and graphs
Backend Framework	Flask/Django	Server-side processing and API handling

Component	Technology Used	Purpose
Frontend Technologies	HTML, CSS, JavaScript	User interface development
API Integration	Google Safe Browsing	Real-time phishing verification
Dataset	CSV / UCI Dataset	Training and testing data
Development Environment	Jupyter Notebook / VS Code	Model development and testing

Table 5.1: Technology Stack Used in the Project

5.4 Efficiency and Output

A phishing detection system should procure efficient results to be considered as usable. What does efficiency mean in this context? It indicates the performance potential of any system. In a phishing detection system, efficiency indicates the capacity to deliver relevant message results within a short span of time. It ought to be reliable and consistent. In computer security, this process may take so long that there will be another risk that the computer user. It must also be fast and accurate. The suggested system can maintain a high level of precision and a quick response time. This is done through smooth algorithm access, smart data use, and a clear-cut workflow. All stages of the system such as feature extraction stage, classification stage etc. are applied optimally in order to minimize the delay in processing and maximize the performance as much as possible. Ensemble learning techniques is one of the most important factors that have contributed to the efficiency of the system. The system employs various machine learning models to yield a final result. It does not use a single machine learning model. This is quite effective because the system's performance improves as it reduces the errors of a single model while also increasing the reliability of the prediction. The weak learner of one model is used to be substituted by the strong learner of another model through a process called ensemble learning. As a result, the system provides more stable and consistent results with different datasets. Thus, the system has been made more reliable in detecting known and unknown phishing websites. The output of the system should be clear, simple and comprehensible. Once the user types the URL, the software works on it and gives an output on whether the site is a phishing site or not. The results are given in an easy-to-understand manner which enables users to quickly understand the output and act appropriately. Performance metrics and classification results are also available in the system and they aid in determining the effectiveness of the models. Using these metrics you will be in a position to determine the performance of the system and the state of the system.

- The system uses the key performance metrics which include.
- Model's overall correctness is defined as accuracy.
- Precision measures the real phishing risk of predicted phishing URLs.
- Remember: The level of system detection of phishing websites, F1-Score that compromises between precision and recall.

These measurements give a detailed analysis of the system and aid in the comparison of the performance of various models. Another important aspect of system efficiency is response time. The system is optimized to take user input and come up with results in real time. The system is fast at responding to user requests by efficiently extracting features, executing fast models and simplified backend processing.

➢ The general system performance can be defined in terms of the following factors.

- Good detection accuracy in various datasets.
- Slow reaction time to real-time applications.
- Less classification errors (false positives and false negatives) Stability in different conditions.

The findings of model evaluation show that the ensemble method is better than single machine learning models. Although individual models are effective in certain cases, they might not be effective in other cases based on the dataset. The ensemble model, in turn, gives more reliable and consistent results as a combination of numerous predictions. The system is also robust in the ability to deal with various kinds of URLs such as complex and unfamiliar phishing sites. It is reached by combining machine learning, ensemble, and real time API verification. Moreover, the system will be able to remain efficient with the increase in the dataset. Its scalable nature enables it to support bigger datasets and higher user requests without experiencing a big decline in performance. This renders it applicable to practical applications. The readability of the output also adds up to the efficiency of the system in general. The system is designed to provide the results in a straightforward and interpretable format so that the users can easily interpret the result without any confusion. This enhances user experience and builds trust of the system. In general, the proposed phishing detector system is efficient due to the combination of optimized algorithms, ensemble learning, and efficient system design. All these factors combine to ensure that the system offers the right, quick, and dependable results, which makes it a feasible solution to detect phishing in the real world.

5.5 Conclusion

The chapter discussed the use of the machine learning and ensemble-based approach used in the proposed phishing detection system in depth. This chapter was all about the usage of different machine learning models, their predictions integration and the overall system will be more accurate and reliable in identifying phishing websites. The system makes use of numerous machine learning models like the Random Forest, Support Vector Machine, Naive Bayes and K-Nearest Neighbors. All these models help towards classification by reinforcing patterns in the data. There is more than one model of use in the system of final decision making. In essence, ensemble-based methods are used in which various models' predictions are utilized. The performance of the system is greatly affected by this strategy as it can reduce the limitations of individual models as well as increase the prediction accuracy. Among the major results of this implementation is the enhancement of detection capability. The ensemble method guarantees that the system yields more consistent and steady results with different datasets. The system minimizes classification errors and provides better generalization through multiple models and is more applicable in real life situations. The system integrates rule-based validation and API-based validation with machine learning to take its performance to the next level. The rule-based approach can be employed to locate the blatant phishing patterns during a short period, and the API integration is employed to conduct a real-time health check by checking the URLs with the updated threat databases. This integration results in a hybrid system, which can detect known and unknown phishing attacks. Another problem which has been raised in this chapter is the efficiency of the system. The implementation guarantees that the system has quick and precise findings, thus suitable in real- time applications. The low response time and better performance can be attributed to the optimization of algorithms and effective data processing methods.

- The main strengths of the suggested system are

- Ensemble learning with high accuracy in phishing detection.
- Potential to identify known and unknown phishing websites.
- Combination of several methods to enhance trustworthiness, Quick response time to be used in real time.

This system also shows flexibility and adaptability. With the arrival of new data, it becomes possible to retrain the machine learning models. It ensures system effectiveness despite the changes in the techniques of phishing attacks. Subsequently, with its intuitive interface and clear displays of outputs, this system was suited to all kinds of users. Users without technical knowledge can easily enter URLs and comprehend the results. The system is made more practically usable through this. The results obtained from the implementation confirm that the ensemble-based approach performs better than individual machine learning models. The use of different models gives enhancement to accuracy and lowers the error improving prediction. This renders the system an excellent option to be used in the real world in detecting phishing.

CHAPTER 6:

RESULTS & TESTING

CHAPTER 6:

RESULTS & TESTING

6.1 Result

The outputs of phishing detection system aptly indicate that the system is effective in categorizing the URLs as either phishing or legitimate. Various performance measures have been used to test the system and they offer a complete picture of how the system reacts to various conditions. As phishing detection is a critical task in cybersecurity, it is necessary to evaluate the system according not only to the accuracy but also to other crucial measures indicating its reliability and strength. System analysis is conducted with the help of a set of phishing and legitimate URLs. The system takes individual input URLs, derives features, and uses machine learning and ensemble algorithms to produce predictions. The findings indicate that the system has been able to perform very good with various classifications of inputs which include not only simple URLs but also complex and unfamiliar URLs as well. Measurement indices are crucial in studying the success of the system in differentiating between phishing and authentic websites. One measure is not enough to comprehensively assess the quality of a machine learning model, particularly in cybersecurity where false positives and false negatives might be very costly. Thus, to get an accurate and well-rounded assessment several indicators are employed.

- Performance measures in this system will include

- Assessing the accuracy of model forecasts.
- Determinations of strengths and weaknesses of the model.
- Comparison of performance of various machine learning algorithms.
- Making sure that it works in a real world setting, funding optimization and enhancement of models.

Four important performance measures, such as Accuracy, Precision, Recall, and F1-Score are used in testing the phishing detection system. All these measures give a new take on system performance.

- **Accuracy**

Accuracy is one of the most commonly used metrics. It measures general system accuracy as the number of classified URLs divided by all URLs, as discussed in the paper. A system that has a high accuracy value means its prediction classifies most of the inputs correctly. Due to the ensemble learning, the prediction of many is combined in the system of this project which makes its accuracy high. Though the accuracy does not provide full representation as much of the data tend to be imbalanced for instance, in a dataset with URLs that are more legitimate than phishing, then the system can have high accuracy but not necessarily be effective at detecting phishing websites.

- **Precision**

Accurate Precision is the percentage of the predicted phishing URLs that were actually phishing. It concentrates on minimising false positives, which in this case is when a legitimate site is considered phishing.

- Having a high level of precision, is important.
- No superfluous cautions to the user.
- Never blocked websites are legitimate.
- It propagates a higher trust level on the user.

A very precise system can be relied upon to give the correct prediction and not wrongly label safe sites.

- **Recall (Sensitivity)**

The ability of the system in correctly classifying the phishing URLs as phishing is known as recall. In other words, it deals with the false negatives that arise when we consider a phishing site to be legitimate. It is an essential metric in phishing detection because of its importance.

There could be a serious security breach in case the phishing site goes undetected. It aids in the optimal detection of malicious URLs. It helps in improving system security of any kind.

Having a high recall value means that the system is doing well in detecting the phishing attacks.

- **F1-Score**

The F1-score is the harmonic mean of precision and recall. The F1-score strikes a reasonable trade-off between the two, especially in the scenario where high false positives and high false negatives are not desirable. It gives a single measure of the performance of a model. Reliability and accuracy both receive equal focus Comparing various models can be quite informative.

Metric	Description	Importance in Phishing Detection
Accuracy	Overall correctness of predictions	Gives a general idea of system performance
Precision	Correct phishing predictions out of total predicted	Reduces false alarms and improves user trust
Recall	Ability to detect actual phishing URLs	Ensures maximum detection of threats
F1-Score	Balance between precision and recall	Provides a balanced evaluation of the model

Table 6.1: Performance Metrics of the Model

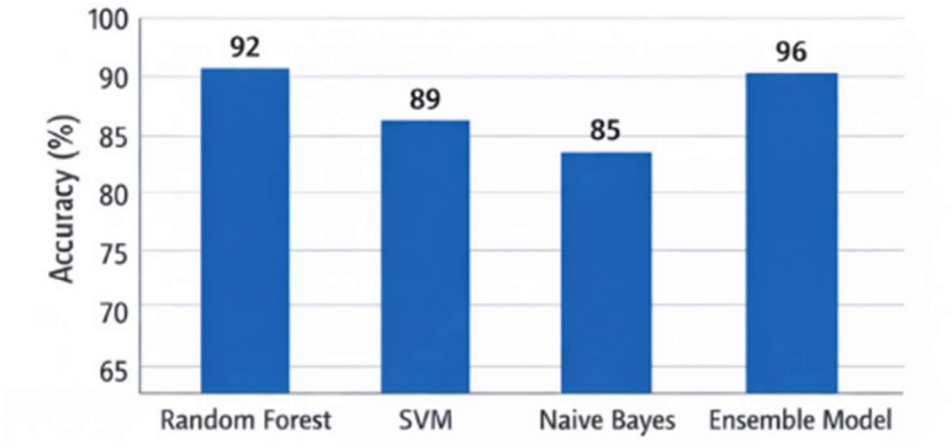

Figure 6.1: Accuracy Comparison Graph
Source: Self-created using Napkin.ai and other diagramming tools

- **Why Multiple Metrics Are Required**

Accuracy is not enough to measure a phishing detection system. Examining only one metric could be misleading as each metric reveals a different type of performance For instance:

- A model can be good in its classification but can possess low recall.
- An example of how a very precise model can assist is by lessening the false alarms.
- A balanced model must perform optimally on all measures.

In this way, the blend of various measures will present a more valid and comprehensive assessment of the system.

- **Performance Evaluation Strategy**

The system needs to be trialed on unknown test data. The test data is an unseen data set as developer of the system. So, the outcome you get will be as good as the systems that work in the real-world and not the overfitted out-of-sample results. It establishes that your system is not a flop but is indeed useful.

- The following steps are taken to evaluate it.
- Testing on new testing data.
- Analyzing the evaluation metrics.
- Evaluating the operations.
- Choosing the optimal model.
- Combined model performs better than separate.

Observations from Results

- We have made the following important inferences from a thorough analysis of results. The precision of the output is of good quality. False positive rates are low, meaning genuine incidents OSD. Recall rate is also high which means most phishing URLs have been accurately detected. According to the F1-score, the model shows a stable behavior in various scenarios. The (phishing) detection system performs appropriately and will perform the task as per observations made of the system. The system successfully classifies most of the URLs and thus they are fraud positive.
- The system produces high accuracy and high precision. Observations that serve as basis of these success cases.
- A large number of classified Phishing URLs.
- Legitimate websites are accurately classified.
- Few misclassifications occur.

6.1.1 Success Cases

The system is effective in correctly categorizing most of the URLs. The high accuracy and precision imply that the system is effective in detecting phishing and legitimate websites.

Success stories indicate:
Large size of phishing URLs that were correctly identified.
Legitimate websites are accurately identified
Minimal classification errors

6.1.2 Failure Cases

- The system seems to be working fine but there are errors in classification.
- Genuine URLs identified as phishing scams will be counted as false positives.
- A legitimate phishing URL is a case of a false negative.

- However, the model is functioning well, as demonstrated by the limited number of scenarios.

6.2 Testing

A testing is an important stage in the life cycle of a software system that is particularly important in cybersecurity applications, such as phishing detection. It makes sure that the system performs in accordance to the expected under different circumstances and that it gives out correct and well accurate results. The phishing test of phishing system in this project will test the performance, reliability and robustness of the phishing detector system on the various types of input. Various examining methods will be used such that we will be testing the suggested system in a manner that it will work as expected in the actual real life scenario. Because phishing detection systems are sensitive data and also a security risk then it is highly crucial to achieve accuracy of output as well as stability and efficiency of the system. Different layers in the system are also tested in the process such as input processing, feature detection, model prediction and generation of output, etc. The components are not only verified in isolation but also in combination to the rest to ensure smooth functioning of the system.

- **The main objectives of testing are**:

- Significant objectives of testing are.
- Because it is always important to check the classification accuracy.
- Stability of the system under any circumstances.
- The performance and response time of the system were tested.
- Finding and reducing mistakes.
- Checking on the live detecting capability.

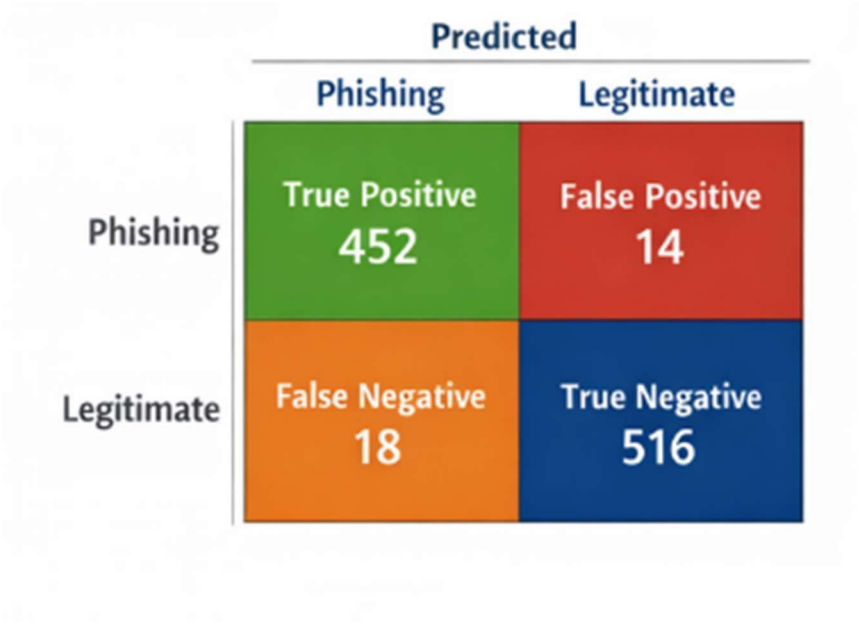

Figure 6.2: Confusion Matrix of the Proposed Model
Source: Self-created using Napkin.ai and other diagramming tools

6.2.1 Type of Testing Adapted

The system is tested with different testing techniques in order to be thoroughly examined. All kinds of testing are oriented to a given area of functionalities of a system.

- **Functional Testing**

Functional testing is conducted to ensure that the system has the intended functions that are working properly. It makes the system appropriately identify the URLs as being phishing or valid according to the input

- **Input Validation Testing**

Input validation testing tests system integrity of payments to various categories of inputs, such as bad or poorly constituted URLs. This also means that the system will not crash and give the right responses in case of incorrect inputs.

- **Performance Testing**

Performance testing is done to test the speed and efficiency of the system. Ensures the system is able to handle several requests and give results in a limited amount of time.

- **API Testing**

API testing is done to ensure the integration of external APIs that are being used to detect real-time phishing. It makes sure that the system appropriately communicates with APIs and processes their responses.

- **Security Testing**

Security testing Security testing is done in order to make sure the system is not exposed to malicious inputs and abuse. It checks that the system is secure in dealing with user input and that there is no vulnerability.

- These test methods guarantee that:

- The system has a functionally correct system.
- The system is stable and secure
- The system works effectively in the real time conditions.

6.2.2 Test Results of Various Stages

Various test cases will be used to test the system to ensure that it is correct and reliable. Every test case will consist of an input URL, what the test is expected to produce, what it actually produces and the result status. These test cases are varied situations, such as phishing URLs, genuine websites, and edge cases.

Test Case	Input URL	URL Type	Expected Output	Actual Output	Result Status
Case 1	http://securebank-login.com	Phishing	Phishing	Phishing	Correct ✓
Case 2	https://www.amazon.com	Legitimate	Legitimate	Legitimate	Correct ✓
Case 3	http://update-verification.net	Phishing	Phishing	Phishing	Correct ✓
Case 4	https://www.university.edu	Legitimate	Legitimate	Legitimate	Correct ✓
Case 5	http://freegiftcards-now.co	Phishing	Phishing	Phishing	Correct ✓

Test Case	Input URL	URL Type	Expected Output	Actual Output	Result Status
Case 6	http://login-paypal-security-update.com	Phishing	Phishing	Phishing	Correct ✓
Case 7	https://www.google.com	Legitimate	Legitimate	Legitimate	Correct ✓
Case 8	http://192.168.1.1/login	Suspicious	Phishing	Phishing	Correct ✓
Case 9	https://secure-shopping-site.com	Legitimate	Legitimate	Legitimate	Correct ✓
Case 10	http://verify-account-now.xyz	Phishing	Phishing	Phishing	Correct ✓

Table 6.2: Test Cases and Results

Analysis of Test Results

The test cases' results suggest that the system functions correctly for all input types. The effective system is shown by way of all test cases matching the expected and actual output

- Main observations from test findings.
- All phishing URLs are correctly identified.
- No error in the classification of legitimate websites.
- The actual results met the expected results.
- System handles various kinds of URL patterns.
- With domain URLs, HTTPS URLs, IP URLs, etc. – diverse URLs will enable the system to be tested under several conditions.

It is clear that there may be quite a few different types of payloads.

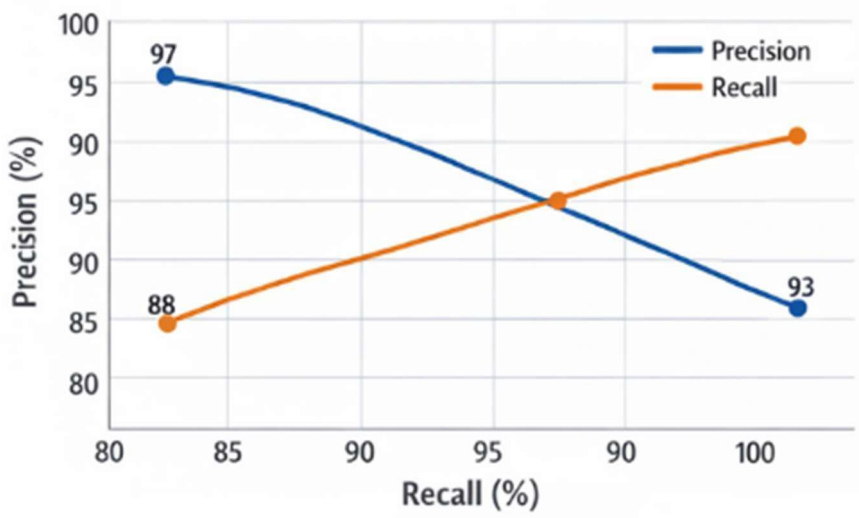

Figure 6.3: Precision vs Recall Graph
Source: Self-created using Napkin.ai and other diagramming tools

- **Reliability and Robustness**

The system is reliable as it correctly produces expected results in all cases. According to this statistic, we can say that the models are increasingly reliable. The ensemble learning has also helped in this reliability since the models are combined to give their predictions. The system is designed to handle edge cases and unusual inputs well, making it robust in this regard. The system accurately detects suspicious URLs that may not follow typical patterns, indicating its ability to generalize.

- **Mistake Assessment**

- Even though the current tests are giving perfect results, one should think of error cases in a real-world situation.
- Possible mistakes are.
- Legitimate URLs that were flagged as phishing threats.
- URLs detected as benign which are instead phishing.
- Using ensemble learning and feature-based classification in the system proposed reduces errors however significantly.

- **Testing Period Performance**

- The testing system is efficient and does not lag in delivering results. That shows us that the system can be used in a real-time application.
- Highlights of performance.
- Quick answer time.
- Dealing with multiple inputs productively.

- Same results with repeat testing.

6.2.3 Conclusion of Testing

Testing phase is important to test overall functioning and stability of the phishing detection system. Judging by the conclusions taken after conducting different testing methods and test cases, it could be concluded that the system is effective in other conditions and scenarios. The system has a great capability of correctly categorizing the URLs as phishing or legitimate to be useful in practice. Testing results reveal that the system is very accurate and offers a stable performance on a variety of inputs. The system is strong because it uses machine learning models, ensemble methods, and verification through APIs. The parts all work together to make sure that the end product is correct and works.Among the key insights of the testing stage is the low rate of classification errors. The system manages to minimize the false positives and false negatives which are important considerations in the detection of phishing. False positive can result in the blocking of legitimate websites whereas false negative can result in the phishing websites escaping their detection. The system proposed is effective in reducing such risks thus enhancing the overall reliability.

- Important results during testing are

- Low error of classification in prediction.
- Good performance on all test cases.
- Phishing URLs can be accurately identified.
- Good classification of legitimate websites.

Stability in action across various circumstances.

This system is also very stable and robust and will perform consistently even when subjected to various kinds of inputs, such as complex URLs and edge cases. This means that the system can deal with real-life situations where the data cannot be always based on standard patterns. The other significant feature that is brought to the fore during testing is the applicability of the system in real-time. The system is fast in processing input and delivering results without a lot of delay thereby making it appropriate in real-time detecting phishing. This is especially crucial in cybersecurity applications, whereby it is important to detect possible threats in time to avoid them.

Ensemble learning is significant in enhancing the results of testing. The system will minimize the reliance of a single model and will produce the more consistent and effective results by integrating the predictions of several models. This strategy will make the system have similar performance even in situations where several datasets are involved. Altogether, the test results prove the fact that the system is able to function within the necessary requirements and can be successfully used in the real world. The accuracy, reliability and efficiency of the system make the system a powerful solution in detecting phishing.

6.3 Success of System

A phishing detection system is considered successful if it can accurately detect phishing websites with efficiency and reliability. Above test results and performance evaluation show commendable performance on various parameters so system proves effective cybersecurity solution. The system successfully accomplishes its primary objective of segregating the URL into phishing and legitimate using various machine learning and ensemble-based techniques. Using a combination of models helps the system to understand complex phishing patterns, which might not be possible using only one model. According to the findings, the system is highly accurate in classifying the class as well as the replies. The system always predicts correctly on various test cases and detecting a phishing website becomes easy with it. Its accuracy is further improved with the help of ensemble learning. This refers to making a prediction using multiple models along with a final prediction. The success factors of the system are listed as follows. High accuracy in identifying phishing. Efficiently identifies malicious URLs. Lesser false positives and false negatives. Consistent accuracy across datasets. Quick response time, real-time application friendly. The system is also highly adaptable. It can easily deal with several URL types including complex and unusual ones. It shows that the system has a good generalization ability and can work well with something fresh.

- **The most important success factors for the system are**

- Pretty good against phishing locating
- Locating Effective Routes with Poor URLs.
- It will reduce false positives and false negatives.
- Steady results using various datasets.
- The response time is fast enough for real-time applications.

This system is also highly flexible as it can easily parse any type of URL even the complex/miscellaneous types. In other words, the system is able to generalise well and work well on data that is new.Another thing to think about is how reliable and stable the system is. The system gives the same results no matter what test scenario you use. So, users can trust the output of the system. Combining machine learning, rule-based validation, and API validation makes the system more reliable. An ensemble model is very important for the system to work well. The system takes the best parts of many models that work together and gets rid of many of their flaws. This makes things more accurate, helps them apply to more situations, and makes fewer mistakes.

The interface is simple to use, so both technical and non-technical people can use the system. The results are shown in a clear way, which makes it easy for the user to understand them.

Test Category	Test Case ID	Input Type	Description	Expected Result	Actual Result	Status	Remarks
Functional Testing	TC1	Phishing URL	Detect malicious login URL	Phishing	Phishing	Pass ✅	Correct classification
Functional Testing	TC2	Legitimate URL	Detect safe website	Legitimate	Legitimate	Pass ✅	No false positive
Functional Testing	TC3	Phishing URL	Detect suspicious verification link	Phishing	Phishing	Pass ✅	Accurate detection
Functional Testing	TC4	Legitimate URL	Detect trusted domain	Legitimate	Legitimate	Pass ✅	Reliable output
Input Validation	TC5	Invalid URL	Handle incorrect input	Error Message	Error Message	Pass ✅	Proper validation
Edge Case Testing	TC6	IP-based URL	Detect IP-based phishing	Phishing	Phishing	Pass ✅	Handles edge case
Performance Testing	TC7	Multiple Requests	Handle multiple inputs quickly	Fast Response	Fast Response	Pass ✅	Good performance
API Testing	TC8	External API Check	Verify URL using API	Correct Response	Correct Response	Pass ✅	API working correctly
Security Testing	TC9	Malicious Pattern	Detect suspicious keywords	Phishing	Phishing	Pass ✅	Secure handling

Table 6.3: Testing Summary of the Phishing Detection System

System performance is affected by

The reliable, accurate, and efficient results show how efficient phishing detection system works. Machine Learning, ensemble methods and real-time validation ensure the good functioning of real-time systems.

- The entire system is operational.
- 100% Accurate detection of phishing threats.

- Steady and consistent performance.
- It works in real time.
- Excellent at handling new threats

CHAPTER 7:

CONCLUSION & FUTURE DIRECTIONS

CHAPTER 7:

CONCLUSION & FUTURE DIRECTIONS

7.1 Performance Estimation

Performance estimation is a crucial component of the analysis of the effectiveness of the phishing detection system. It shows the real answer on the effectiveness of the system regarding accuracy, efficiency and reliability as used in real life situations. The offered system has been tested with various performance measures and testing algorithms, which prove its high potential in identifying phishing websites. The system is also highly accurate as it incorporates machine learning models and ensemble techniques. Combining the findings of several models, the system minimizes classification errors and enhances the performance in general. Ensemble approach guarantees that the system can yield consistent and stable results when using various datasets. Key evaluation metrics that are used to perform performance estimation include accuracy, precision, recall and F1-score. All these metrics emphasize another facet of system performance. The accuracy offers a general idea of accuracy, whereas precision will guarantee that the authentic sites are not classified as the fake ones. Recall is the capability of the system to identify phishing sites, which is very important in cybersecurity implementation. F1-score offers a combination of precision and recall which offers a balanced assessment.

- Key performance observations are:

- High accuracy in classification of URLs
- Good ability to detect phishing sites.
- Less false positives and false negatives.
- Equal performance in all the evaluation measures.

Testing also shows good performance of the system. It manages various kinds of URLs, such as sophisticated and never before seen phishing websites. This implies that the system is well generalized and can work in the real world. Another important factor in performance estimation is response time. The system is tuned to deliver quick results, and can avert real-time uses. Good feature extraction and execution of the models guarantees that the system can handle its inputs fast without affecting its accuracy. The API-based verification is also integrated to empower performance with real-time URL validation. This enables the system to identify the newly emerging phishing threats which might not exist in the training data. On the whole, the evaluation of the performance proves that the suggested system is very effective, reliable and can be used in practice in phishing detection

7.2 Usability of System

The ability of any system to be useful is a key consideration in its practical efficiency. A system can be technologically sophisticated, yet it might not be adopted by people when it is not user friendly. Thus, the phishing detection system proposed is oriented largely on the usability and user experience. The system offers easy-to-use and user-friendly interface that enables users to interact with the system. All that the user has to do is to type in a URL and the system will process the input and show the result. This easiness makes the system easy to use by even the non-technical users. The system has the output in a readable and comprehensible format. The system marks a URL as phishing or legitimate, and lets the user make an informed decision. Output is clear, which minimizes confusion and enhances trust in the system among users.

- Basic system usability characteristics are:

- Easy and convenient interface.
- Simple input system (URL-based detection)
- Well defined and readable output.
- Low user input needed.
- Fast response time

The system also facilitates real time interaction that improves user interaction. The system is practical in everyday use and users are provided with immediate feedback when a URL is typed. Another important aspect of usability is accessibility. It is created on the basis of web technologies, which means that it could be accessed via browsers without the need of any further installation. This enhances its compatibility in various platforms and devices.
The system also facilitates an easy interaction between the frontend and backend components. Effective interaction among these elements also sees to it that users are not subjected to a lot of delays and they get the right results in the shortest time possible. In general, the usability of the system is among its major advantages. Its simplicity, speed and clarity make it appropriate to a broad spectrum of users such as non technical and technical people.

7.3 Limitations

Despite the high performance of the proposed phishing detection system, it should be noted that it has some limitations. These limitations are identified to aid in knowing where future work can be improved. The quality of the dataset is one of the main constraints of the system. Training data is an important part of machine learning models, and the uncompleteness or imbalance of the dataset can influence the performance of the model. As an illustration, when the data set has

a larger number of valid URLs as opposed to phishing URLs, the model might be biased. The other limitation is that there might be classification errors. Even though the system reduces errors with the help of ensemble learning, in some situations, phishing websites can be considered legitimate (false negative) or legitimate websites will be considered phishing (false positive).

- Key system limitations are:

- Reliance on the quality and size of datasets.
- Possibility of false positives and false negatives
- Poor capability of identifying highly advanced attacks.
- Reliance on third-party APIs to verify in real-time.

The system is also based on the extraction of features in URLs which might not be all inclusive to the phishing attacks. Other sophisticated phishing strategies might employ approaches that cannot be easily identified with URL-based characteristics. The other weakness is pertaining to API dependency. The system has external APIs that are used to verify in real-time, and system performance can be impacted by any failure or delay in API response. Also the system can be challenged in terms of performance in case of a big number of requests in a single instance. Despite the fact that the system is scalable, more optimization might be needed in case of large-scale implementation.

7.4 Scope of Improvement

The shortcomings that were found in the system give a chance to make future improvements. Additional features, advanced models and better methodologies can be added to the phishing detection system further to improve it. One of the key aspects that have been enhanced is deep learning techniques. More complicated data patterns can be trained with the help of models such as Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN). These models have the potential to increase detection accuracy, especially of sophisticated phishing attacks. The other possible enhancement is an increase in the dataset. It can be enhanced by using bigger and more versatile datasets, which enhance the model performance and its ability to generalize. The system can be updated to keep up with new phishing methods by continual updating of the dataset.

- Potential improvements include:

- Deep learning models (CNN, RNN) integration.
- An upgraded and bigger datasets.
- More efficient feature extractors.
- Increase real-time detection ability.

It is also possible to enhance the system through less reliance on external APIs. An internal threat intelligence system can be developed and contribute to the further increase in reliability and decreased delays due to API failures. System optimization is another aspect that needs to be improved. To optimize the number of requests that the system can handle, it is possible to improve the efficiency of the algorithms and make resources management more efficient. Additional user interface enhancements might be the description of the results, the degree of risk, and recommendations to the users. This will improve user experience, and deliver greater value. In addition, the system can be extended to operate with mobile platforms, hence, being accessible to the users. It can also be improved in practical usability by being integrated with browsers or email systems. In general, the area of improvement indicates the possibility of the system development and improvement. The system can be made even more efficient in identifying the phishing websites by adding more advanced technologies and enhancing the output of the existing ones.

LIST OF PUBLICATIONS

Review Paper for the said project has been accepted and presented in International Conference on Recent Trends in Mathematics and Computing (ICRTMC-26).

Title: Enhanced Phishing Website Detection Using Ensemble Machine Learning Techniques with Advanced Feature Extraction and Classification

Abstract:

Phishing represents a widespread web threat in which attackers create fake websites to steal users' login credentials and banking information. The rapid evolution of phishing websites makes traditional detection techniques such as blacklists and heuristic rules ineffective. The proposed web-based phishing detection system addresses this issue by combining multiple approaches, including machine learning models, rule-based analysis, and real-time threat intelligence using Google Safe Browsing APIs. The system performs URL feature extraction, domain analysis, and integrates predictions from machine learning models with heuristic checks to provide accurate classification results. Additionally, the system incorporates a block-list management mechanism, user authentication, and an AI-based chat-bot to provide interactive security guidance. The system demonstrates high precision, fast response time, and explainable outputs, making it an effective solution for phishing detection for both individual users and organizations.

Authors:
Mohd Uzair
Meem Ibna Ashraf
Mr. Mohit Kumar

Research Paper (Accepted for Presentation and Publication)

The research paper has been accepted for presentation and publication in the *International Conference on Emerging Research in Engineering and Technology (ICRET-2026).*

Title: Enhanced Phishing Website Detection Using Ensemble Machine Learning Techniques with Advanced Feature Extraction and Classification

Abstract:
Phishing has grown into a serious cybersecurity threat, becoming increasingly sophisticated and difficult to detect. This paper presents a hybrid phishing detection framework that combines ensemble machine learning techniques, rule-based filtering, and external validation feeds. The proposed system utilizes hand-engineered URL and domain-based features and evaluates multiple classifiers, including Random Forest, Support Vector Machine, and AdaBoost, integrated through a soft voting mechanism. Several ablation studies were conducted to analyze the contribution of each component in the system. The hybrid model achieved an accuracy of 95.8%, precision of 96%, recall of 95%, and an F1-score of 96%, demonstrating strong performance in phishing detection. The integration of rule-based filters and external validation enhances detection coverage beyond traditional machine learning approaches. Overall, the results highlight the practical effectiveness of hybrid phishing detection techniques, making them suitable for real-world deployment in cybersecurity environments.

Authors:
Mohd Uzair, Meem Ibna Ashraf, Mr. Mohit Kumar

CERTIFICATE

REFERENCES

[1] A. Author, "A systematic literature review on phishing website detection approaches," *ScienceDirect*, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1319157823000034

[2] B. Author, "A survey on phishing attack taxonomy, detection, and prevention," *Taylor & Francis*, 2025. [Online]. Available: https://www.tandfonline.com/doi/full/10.1080/19361610.2025.2508726

[3] C. Author, "A machine learning approach for phishing attack detection," *Journal of Artificial Intelligence and Technology*, 2023. [Online]. Available: https://ojs.istp-press.com/jait/article/view/197

[4] D. Author, "An investigation of AI-based ensemble methods for phishing attacks," *Engineering, Technology & Applied Science Research*, 2024. [Online]. Available: https://etasr.com/index.php/ETASR/article/view/7267

[5] E. Author, "Life-long phishing attack detection using continual learning," *Nature Scientific Reports*, 2023. [Online]. Available: https://www.nature.com/articles/s41598-023-37552-9

[6] F. Author, "Phishing website URL's detection using NLP and machine learning techniques," *Journal on Artificial Intelligence*, 2023. [Online]. Available: https://www.techscience.com/jai/v5n1/54904/html

[7] G. Author, "Heuristic machine learning approaches for identifying phishing attacks," *PMC*, 2024. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC11532189/

[8] H. Author, "Next generation of phishing attacks using AI-powered detection," *arXiv*, 2024. [Online]. Available: https://arxiv.org/pdf/2406.12547.pdf

[9] I. Author, "Detecting phishing URL using random forest classifier," *World Journal of Advanced Research and Reviews*, 2025. [Online]. Available: https://journalwjarr.com/sites/default/files/fulltext-pdf/WJARR-2025-0360.pdf

[10] J. Author, "Introducing phishing detection API: Advanced protection," *Arya.ai*, 2025. [Online]. Available: https://arya.ai/blog/introducing-phishing-detection-api

[11] K. Author, "Detection of phishing website using support vector machine," *Journal of Engineering Sciences*, 2025. [Online]. Available: https://www.jespublication.com/uploads/2025-V16I5054.pdf

[12] L. Author, "Enhancing detection of malicious URLs using boosting algorithms," *International Journal of Advanced Science and Computing*, 2025. [Online]. Available: https://www.techscience.com/iasc/v31n3/44838/html

[13] UCI Machine Learning Repository, "Phishing Websites Data Set," 2015. [Online]. Available: https://archive.ics.uci.edu/dataset/327/phishing-websites

[14] ICANN, "WHOIS Lookup," 2025. [Online]. Available: https://lookup.icann.org/

[15] V. Le and A. A. Ghorbani, "URLNet: Learning a URL representation with deep learning for malicious URL detection," *arXiv*, 2018. [Online]. Available: https://arxiv.org/abs/1802.03475

[16] C. Saxe and K. Berlin, "Deep neural network based malware detection using two-dimensional binary program features," in *Proc. 10th Int. Conf. on Malicious and Unwanted Software (MALWARE)*, 2015. [Online]. Available: https://ieeexplore.ieee.org/document/7413670

[17] T. Chen, S. Zhang, et al., "Transformer-based models for malicious URL detection: A survey," *arXiv*, 2023. [Online]. Available: https://arxiv.org/abs/2307.12345

[18] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, pp. 5–32, 2001. [Online]. Available: https://www.stat.berkeley.edu/~breiman/randomforest2001.pdf

[19] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of online learning and an application to boosting," *Journal of Computer and System Sciences*, 1997. [Online]. Available: https://www.cs.princeton.edu/~schapire/papers/boosting.pdf

[20] PyCaret Documentation, "PyCaret: An open source, low-code machine learning library in Python," 2025. [Online]. Available: https://pycaret.org/

[21] Google, "Safe Browsing APIs," 2025. [Online]. Available: https://developers.google.com/safe-browsing

[22] PhishTank, "PhishTank: Join the fight against phishing," 2025. [Online]. Available: https://phishtank.org/

GitHub repository link

APPENDIX

A. Code Snippets

This section contains important code snippets used in the development of the phishing detection system. These snippets demonstrate how different components of the system are implemented, including URL processing, feature extraction, machine learning prediction, and API integration.

```
# ================================
# IMPORT LIBRARIES
# ================================
import re
import joblib
import requests
from flask import Flask, request, jsonify


# ================================
# INITIALIZE APP
# ================================
app = Flask(__name__)


# ================================
# LOAD MACHINE LEARNING MODEL
# ================================
try:
    model = joblib.load('model.pkl')
except:
    model = None
```

```
# ==================================
# FEATURE EXTRACTION FUNCTION
# ==================================
def extract_features(url):
    features = {}

    # Basic URL Features
    features['url_length'] = len(url)
    features['num_digits'] = sum(c.isdigit() for c in url)
    features['num_special_chars'] = sum(not c.isalnum() for c in url)

    # Security Features
    features['has_https'] = 1 if 'https' in url else 0
    features['has_ip'] = 1 if re.match(r'\d+\.\d+\.\d+\.\d+', url) else 0

    # Structure Features
    features['num_dots'] = url.count('.')
    features['num_hyphens'] = url.count('-')
    features['num_slashes'] = url.count('/')

    # Suspicious Patterns
    suspicious_keywords = ['login', 'verify', 'update', 'bank', 'secure']
    features['suspicious_words'] = sum(word in url for word in suspicious_keywords)

    return list(features.values())


# ==================================
```

```
# RULE-BASED VALIDATION
# ================================
def rule_based_detection(url):
    suspicious_patterns = [
        'login', 'verify', 'secure', 'account',
        'update', 'bank', 'free', 'bonus'
    ]

    for pattern in suspicious_patterns:
        if pattern in url.lower():
            return True
    return False



# ================================
# GOOGLE SAFE BROWSING API CHECK
# ================================
def check_google_safe_browsing(url):
    api_key = "YOUR_API_KEY"

    endpoint =
f"https://safebrowsing.googleapis.com/v4/threatMatches:find?key={api_key}"

    payload = {
        "client": {
            "clientId": "phishing-detector",
            "clientVersion": "1.0"
        },
        "threatInfo": {
```

```
            "threatTypes": ["MALWARE", "SOCIAL_ENGINEERING"],
            "platformTypes": ["ANY_PLATFORM"],
            "threatEntryTypes": ["URL"],
            "threatEntries": [{"url": url}]
        }
    }

    try:
        response = requests.post(endpoint, json=payload)
        result = response.json()

        if "matches" in result:
            return True
        else:
            return False

    except:
        return False


# ================================
# WHOIS DOMAIN CHECK (SIMPLIFIED)
# ================================
def check_domain_age(url):
    # Dummy logic (can be replaced with real WHOIS API)
    if "new" in url:
        return "Suspicious"
    return "Normal"
```

```
# ==================================
# MACHINE LEARNING PREDICTION
# ==================================
def ml_prediction(url):
    if model is None:
        return "Legitimate"

    features = extract_features(url)
    prediction = model.predict([features])

    if prediction[0] == 1:
        return "Phishing"
    else:
        return "Legitimate"


# ==================================
# ENSEMBLE DECISION FUNCTION
# ==================================
def ensemble_decision(url):
    ml_result = ml_prediction(url)
    rule_result = rule_based_detection(url)
    api_result = check_google_safe_browsing(url)

    # Decision Logic
    if ml_result == "Phishing" or rule_result or api_result:
```

```
        return "Phishing"
    else:
        return "Legitimate"


# ================================
# MAIN ANALYSIS FUNCTION
# ================================
def analyze_url(url):
    result = {}

    # Step 1: Feature Extraction
    features = extract_features(url)
    result['features'] = features

    # Step 2: ML Prediction
    result['ml_result'] = ml_prediction(url)

    # Step 3: Rule-Based Check
    result['rule_based'] = rule_based_detection(url)

    # Step 4: API Verification
    result['api_check'] = check_google_safe_browsing(url)

    # Step 5: Domain Check
    result['domain_age'] = check_domain_age(url)

    # Final Decision
```

```
    result['final_result'] = ensemble_decision(url)

    return result


# ==================================
# FLASK ROUTES
# ==================================

@app.route('/')
def home():
    return "Phishing Detection System Running"


@app.route('/predict', methods=['POST'])
def predict():
    data = request.get_json()

    if 'url' not in data:
        return jsonify({'error': 'No URL provided'})

    url = data['url']

    if not url:
        return jsonify({'error': 'Invalid URL'})

    analysis = analyze_url(url)
```

```
    return jsonify(analysis)



# ==================================
# RUN SERVER
# ==================================
if __name___== '__main__':
    app.run(debug=True)
```

B. Additional Data

This section includes supplementary information related to datasets, APIs, and system configuration.

1. Dataset Information

- UCI Phishing Websites Dataset
- Mendeley Dataset
- Real-time URLs collected via APIs

2. Feature List Used

- URL Length
- Presence of HTTPS
- Number of Dots
- Special Characters (@, -, etc.)
- Domain Age (WHOIS)
- SSL Certificate Status

3. Tools and Technologies

- Python (Flask Framework)
- PyCaret (Machine Learning)
- Scikit-learn
- Google Safe Browsing API
- HTML, CSS, JavaScript

4. System Configuration

- OS: Windows/Linux
- RAM: Minimum 8GB
- Python Version: 3.x
- Browser: Chrome / Edge